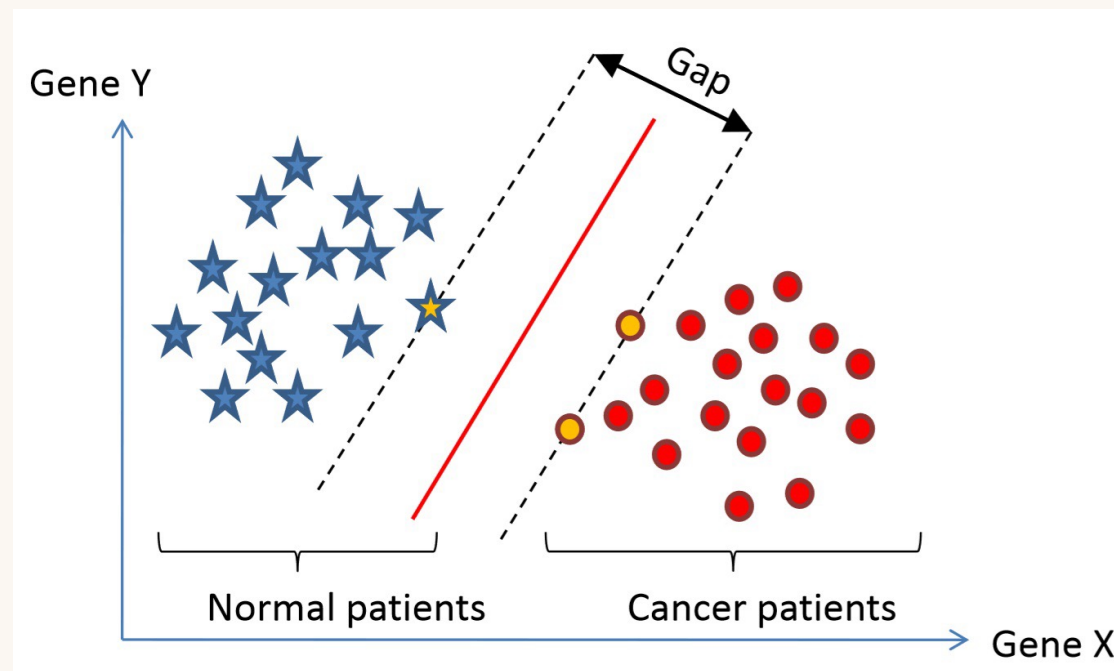


Линейная классификация

логистическая регрессия, SVM, ядра и метрики качества

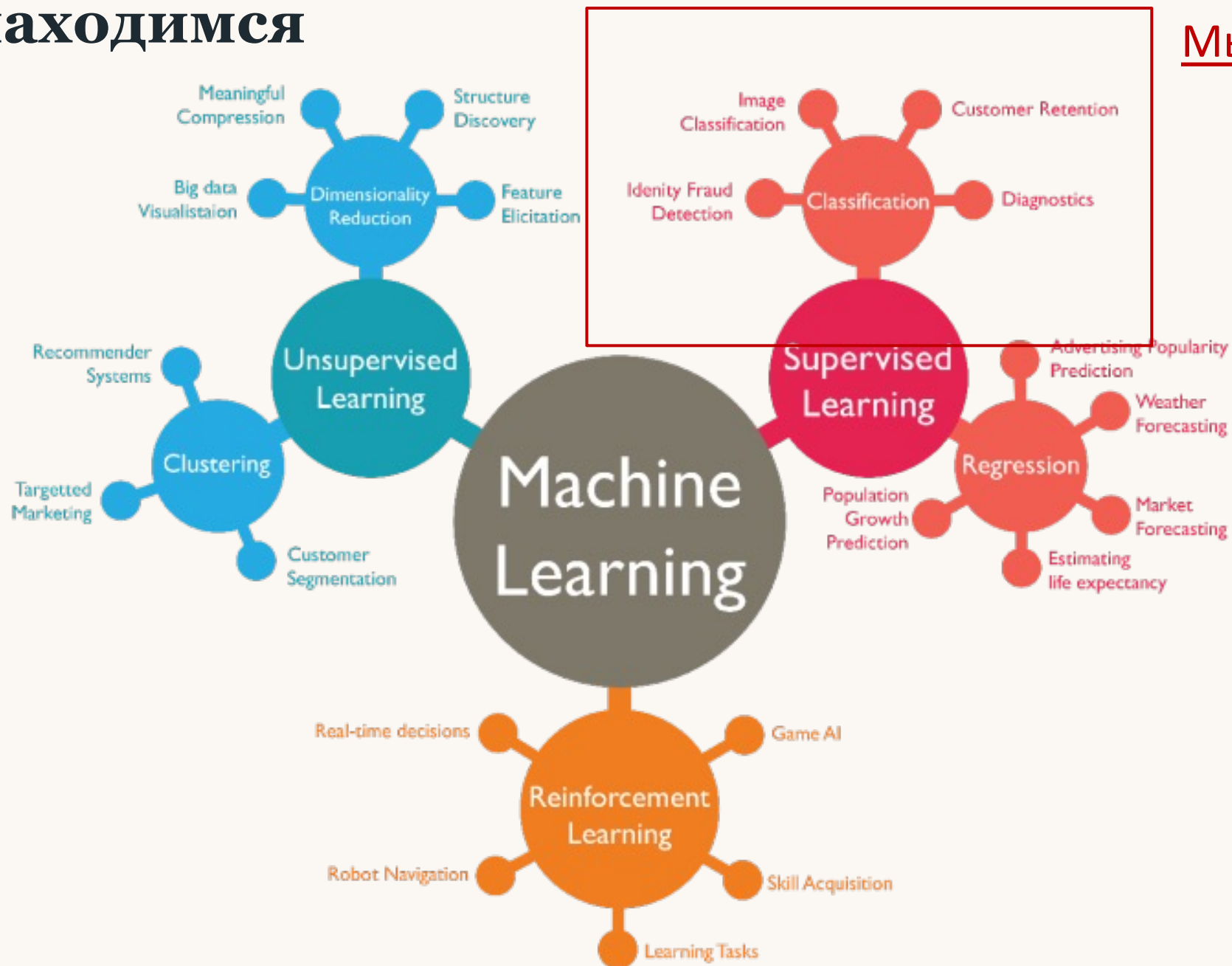
Лекция 3

Полина Полунина · 22 сентября 2026



Где мы находимся

Мы здесь

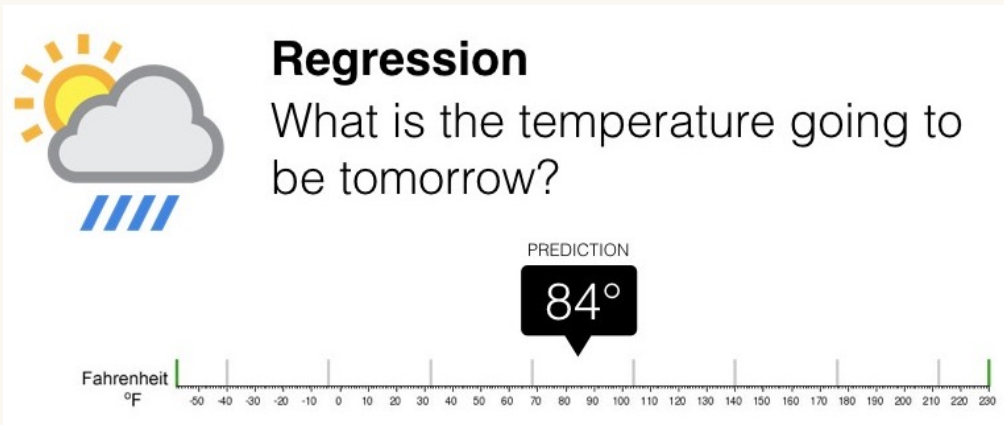


План лекции

- 1 Классификация на пальцах
- 2 Линейный классификатор
- 3 Логистическая регрессия
- 4 Метод опорных векторов (SVM)
- 5 Ядра
- 6 Метрики качества

Регрессия и классификация

Мы здесь



Регрессия: целевая переменная y — число из непрерывного распределения

Задача: предсказать конкретное значение y

Примеры: цена дома, температура в градусах



Классификация: целевая переменная y дискретна

Задача: определить к какому классу относится объект из заранее заданного набора классов

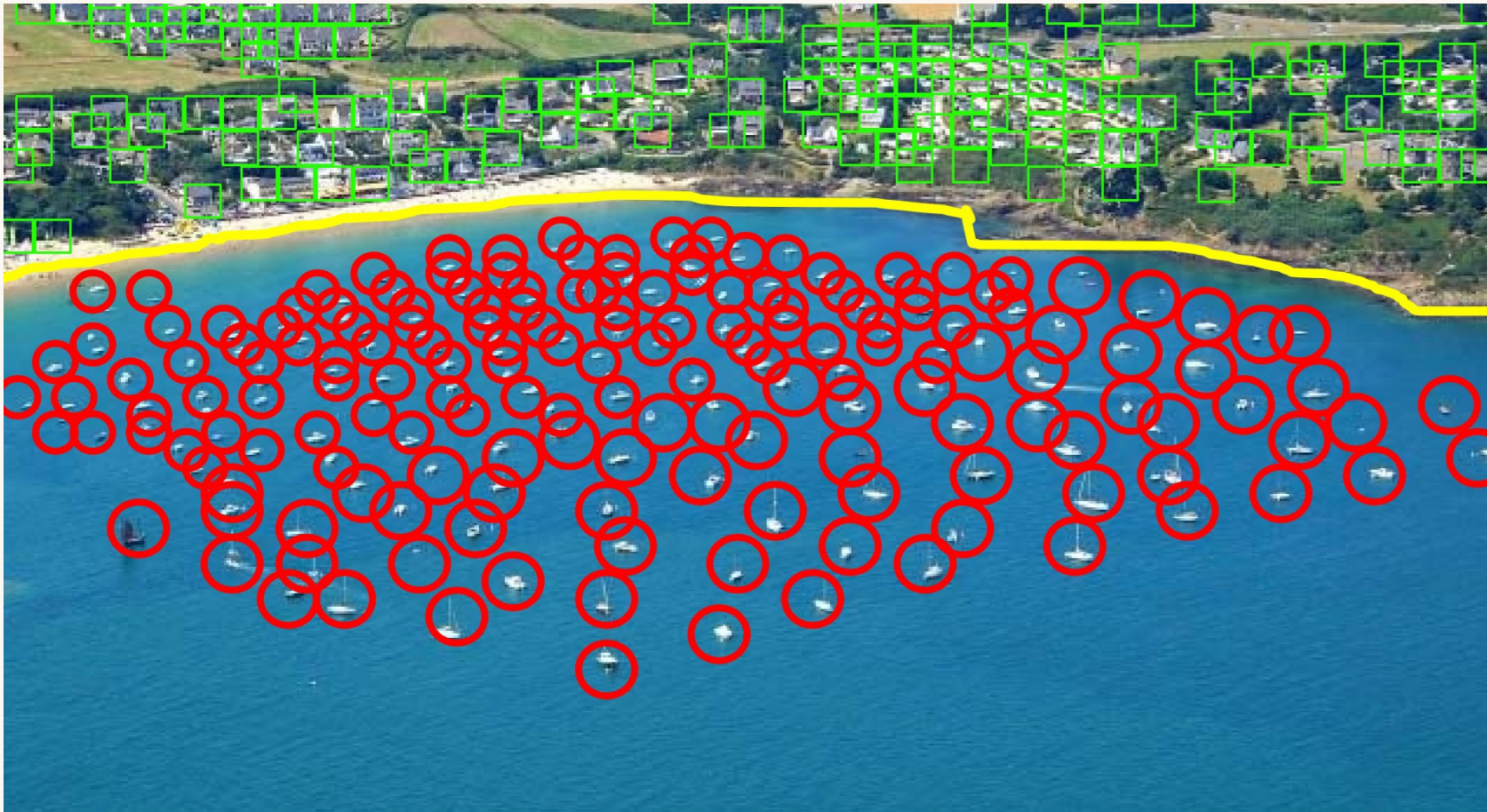
Примеры: «холодно» или «жарко», котик или пёсик

Принципы классификации



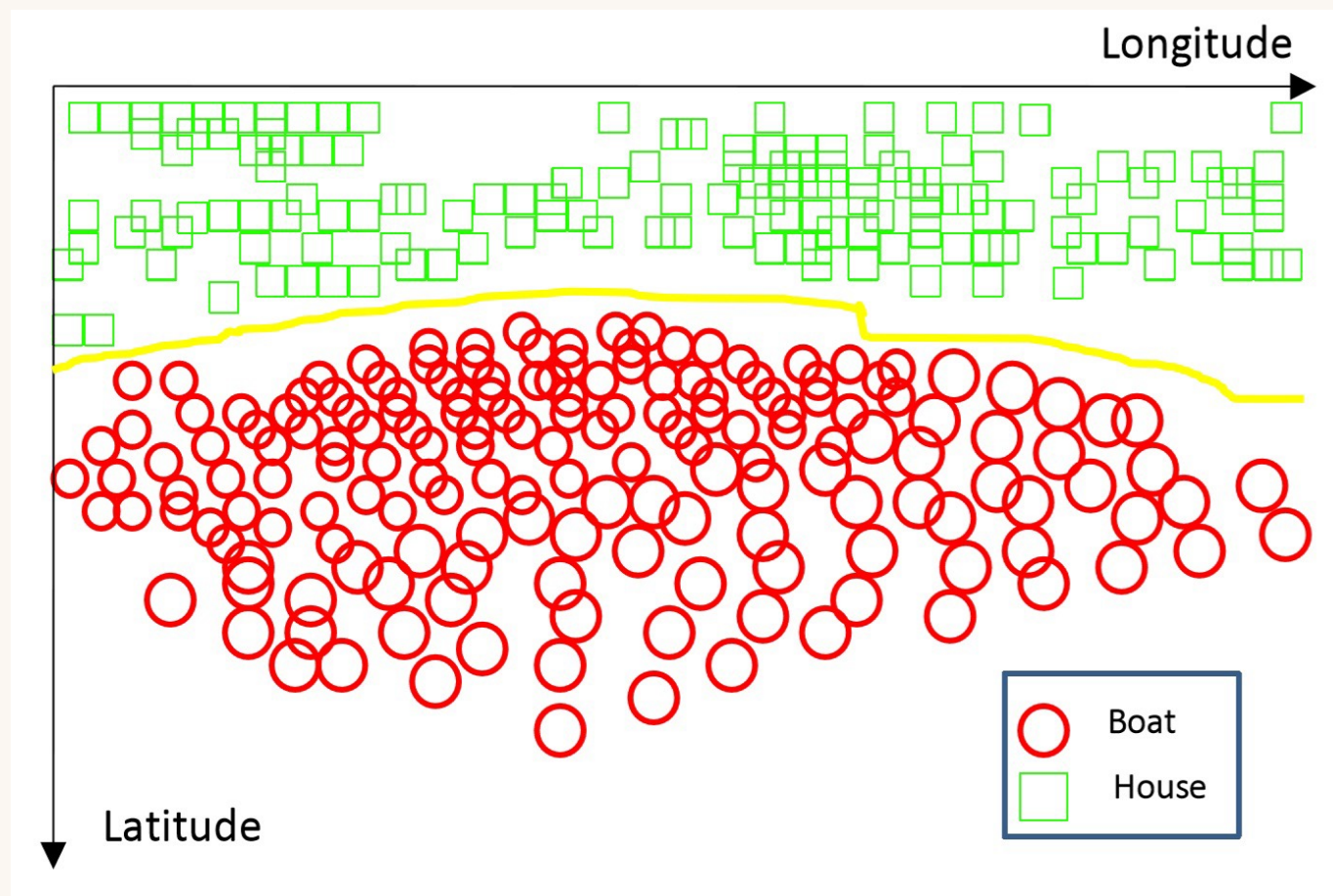
Представим, что объекты на снимке нужно разделить на два класса: «дома» и «лодки».

Принципы классификации



Все объекты выше береговой линии - дома, ниже - лодки. Береговая линия - это **решающая граница** (decision boundary): она разделяет два класса.

Принципы классификации



Алгоритм не «видит» картинку. Сначала описываем каждый объект числами (здесь - координатами на снимке), а затем алгоритм ищет по ним лучшую решающую границу

Котики и пёсики

Пример на всю лекцию: котики сидят на подоконнике, пёсики — на полу.

Хотим научиться отличать одних от других



Край подоконника

Выше края — котики. Ниже — пёсики.

Край разделяет два класса.

Это решающая граница.



Новый пёсик

Этого пёсика раньше не было.
Правило уже есть — смотрим, по
какую сторону края он оказался

**Применяем правило к
новому объекту**

**Это предсказание принадлежности
объекта к классу**



Край ошибается

Этот пёсик забрался на подоконник -
правило назовёт его котиком.

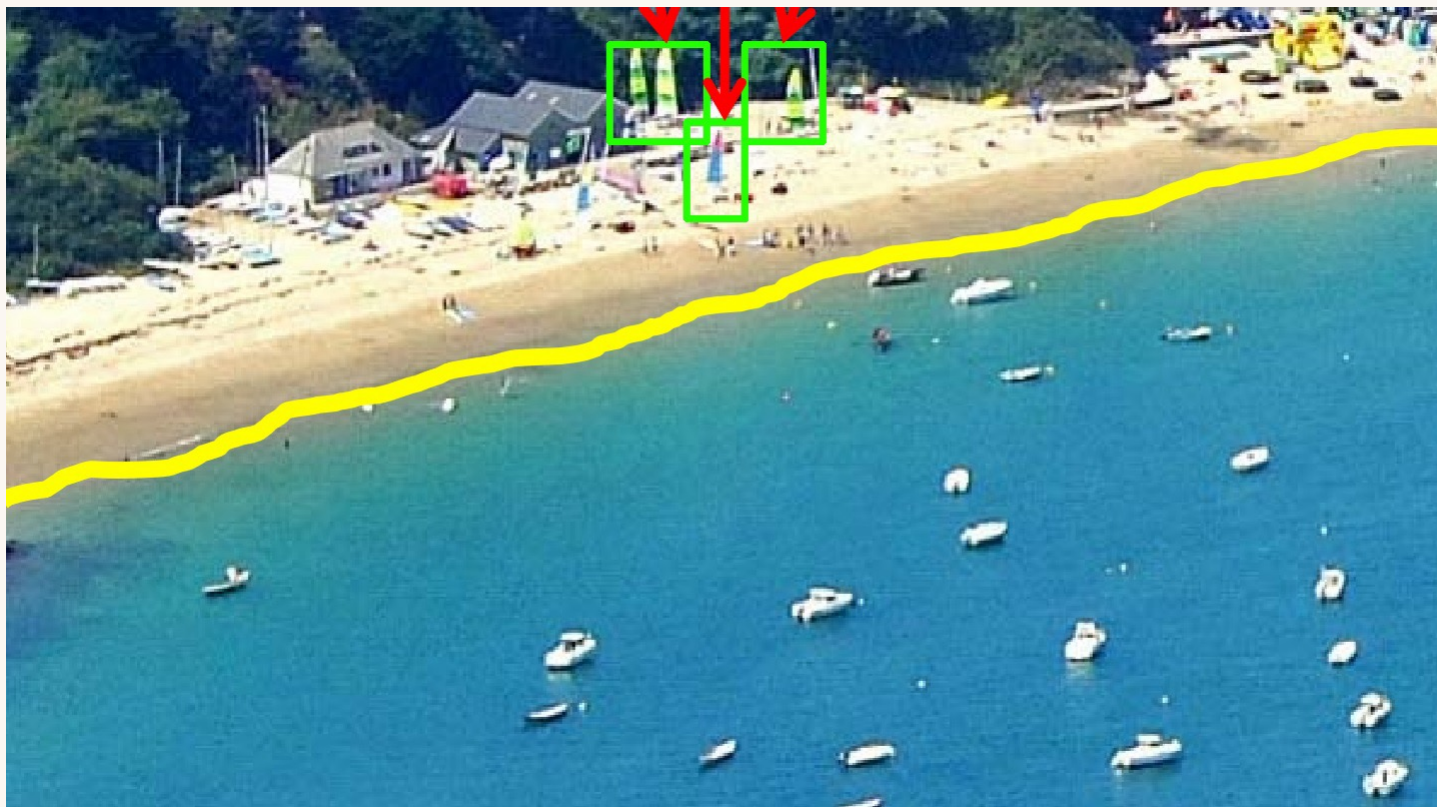
Правило может ошибаться.

Хорошая граница ошибается как можно
реже



В нашем примере с лодками...

Лодки на берегу будут ошибочно классифицированы как «дома» при выборе береговой линии в качестве решающей границы



Котики и собачки: теперь в числах



Опишем каждое животное двумя числами — **признаками**:

- x_1 — длина ушей
- x_2 — длина хвоста

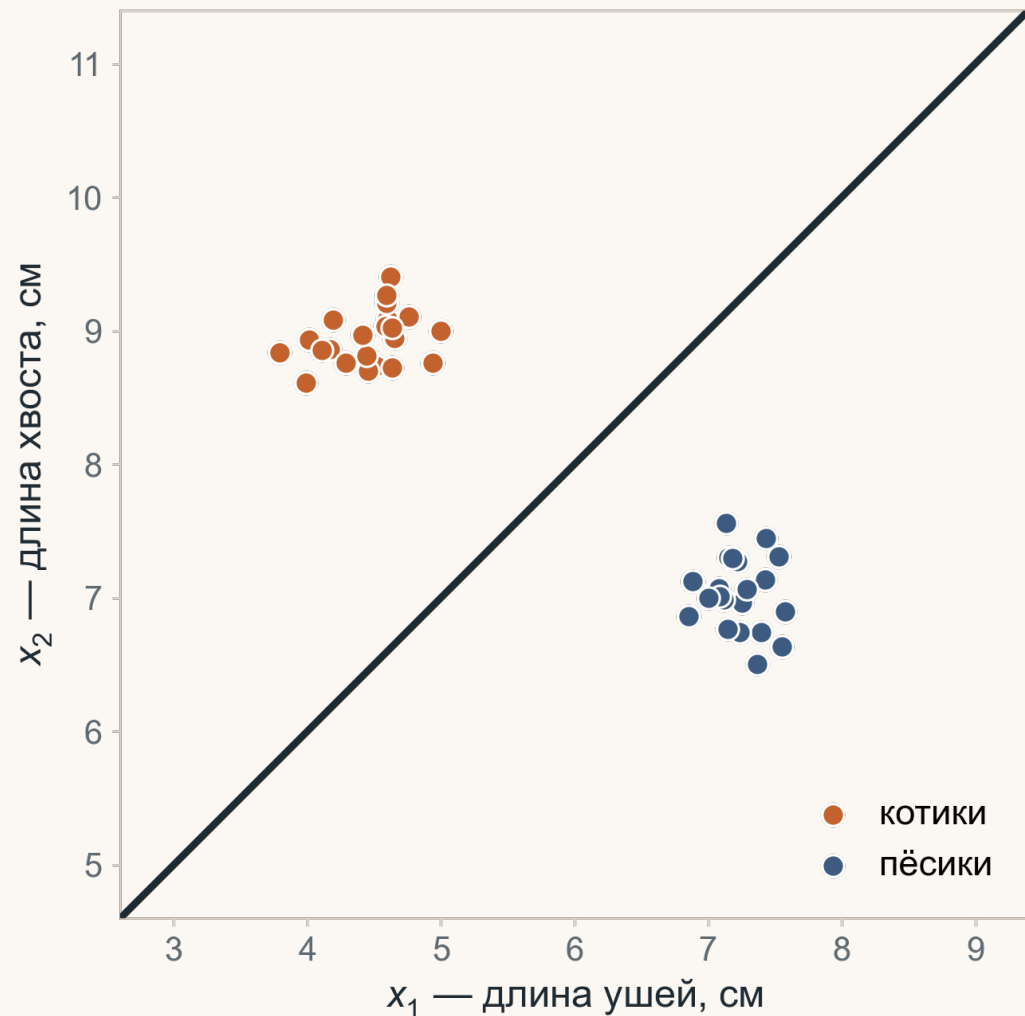
Каждое животное — точка на плоскости, цвет точки — класс.

Признаки — такие же числа, как в регрессии на прошлой лекции. Новое только одно: ответ теперь не число, а класс.

Облака котиков и пёсиков не пересекаются. Попробуем разделить их прямой.

Линейный классификатор

Проводим прямую



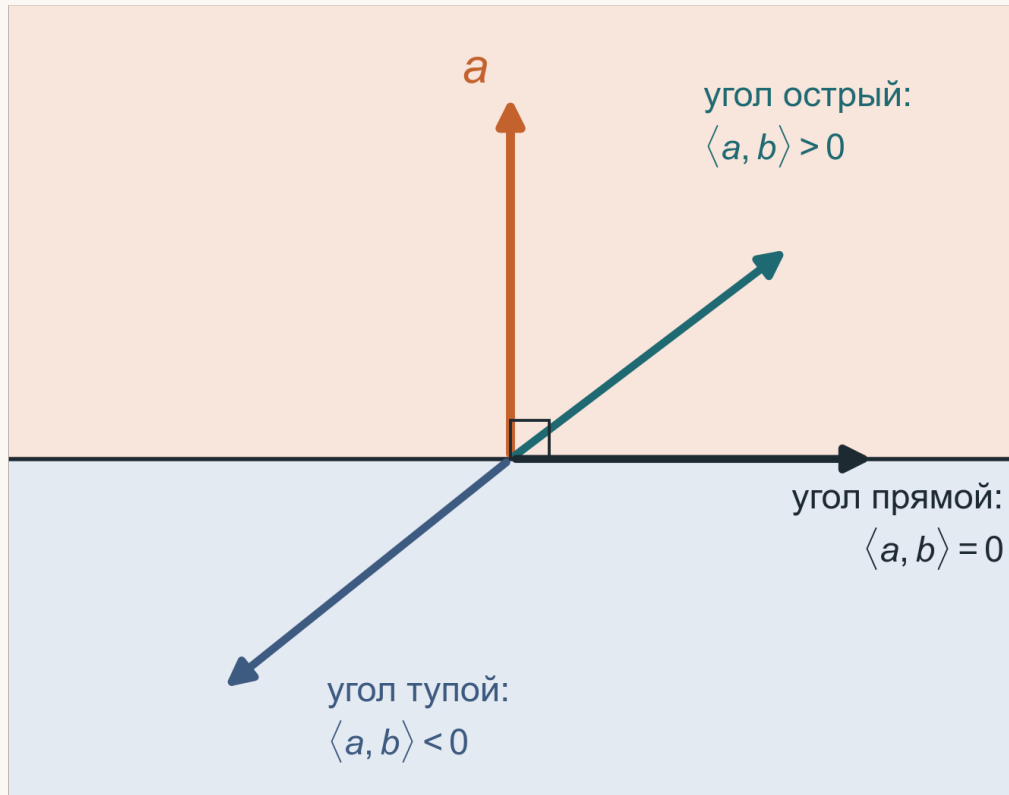
Проведём прямую так, чтобы котики оказались по одну сторону, а пёсики — по другую.

Это тот же «край подоконника», только в координатах.

Чтобы работать с прямой в коде, её нужно записать формулой. Для этого понадобится скалярное произведение.

Скалярное произведение

$$\langle a, b \rangle = a_1 b_1 + a_2 b_2 = \|a\| \cdot \|b\| \cdot \cos \theta$$



$\|a\|$ — длина вектора, θ — угол между векторами.

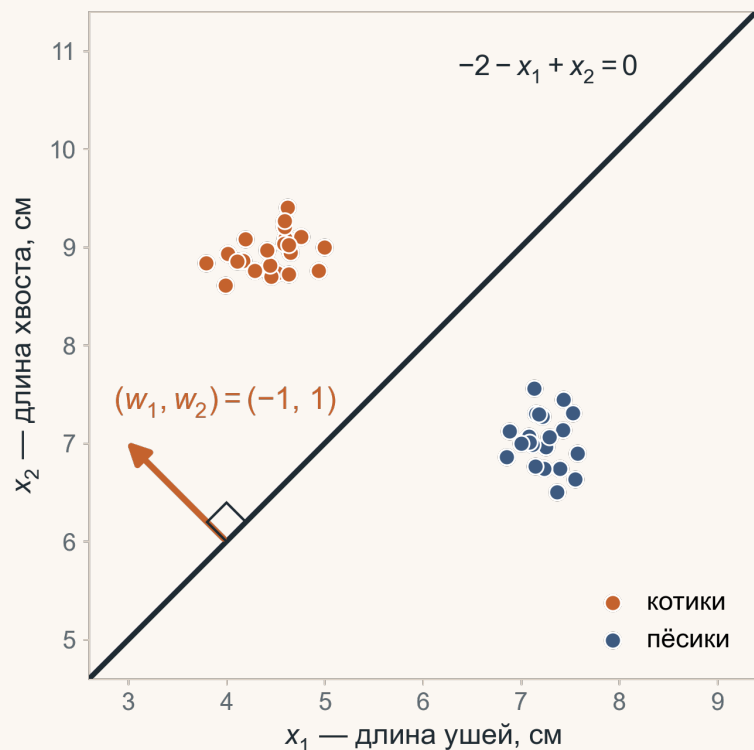
Знак скалярного произведения зависит только от угла: острый угол — плюс, прямой — ноль, тупой — минус.

Пример: $a = (0, 2)$, $b = (3, 0)$. Тогда $\langle a, b \rangle = 0 \cdot 3 + 2 \cdot 0 = 0$ — векторы перпендикулярны.

Уравнение прямой

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \quad \Leftrightarrow \quad \langle w, x \rangle = 0$$

$$w = (w_0, w_1, w_2), \quad x = (1, x_1, x_2)$$



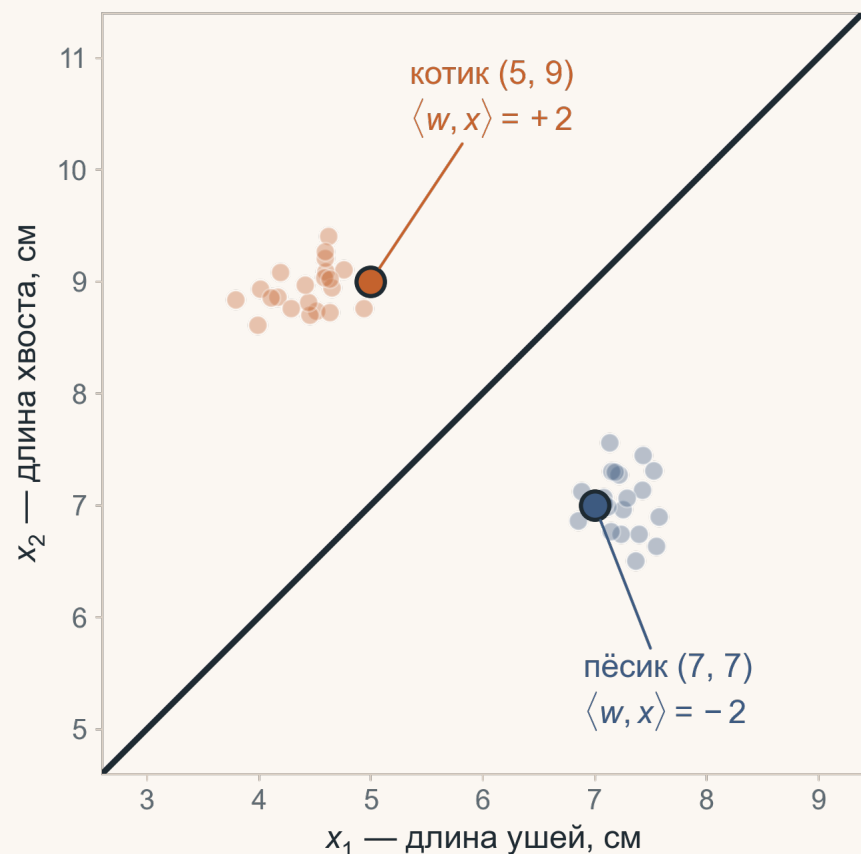
Как на прошлой лекции, добавляем фиктивный признак $x_0 = 1$ — тогда свободный член w_0 входит в скалярное произведение.

Вектор (w_1, w_2) перпендикулярен прямой и задаёт её наклон, а w_0 сдвигает прямую.

Почему перпендикулярен: возьмём на прямой две точки x и x' и вычтем их уравнения: $w_1(x_1 - x'_1) + w_2(x_2 - x'_2) = 0$. Значит, вектор (w_1, w_2) перпендикулярен отрезку между ними.

Как это посчитать

$$\langle w, x \rangle = w_0 \cdot 1 + w_1 x_1 + w_2 x_2$$



Возьмём $w_0 = -2$, $w_1 = -1$, $w_2 = 1$, то есть прямую $x_2 = x_1 + 2$.

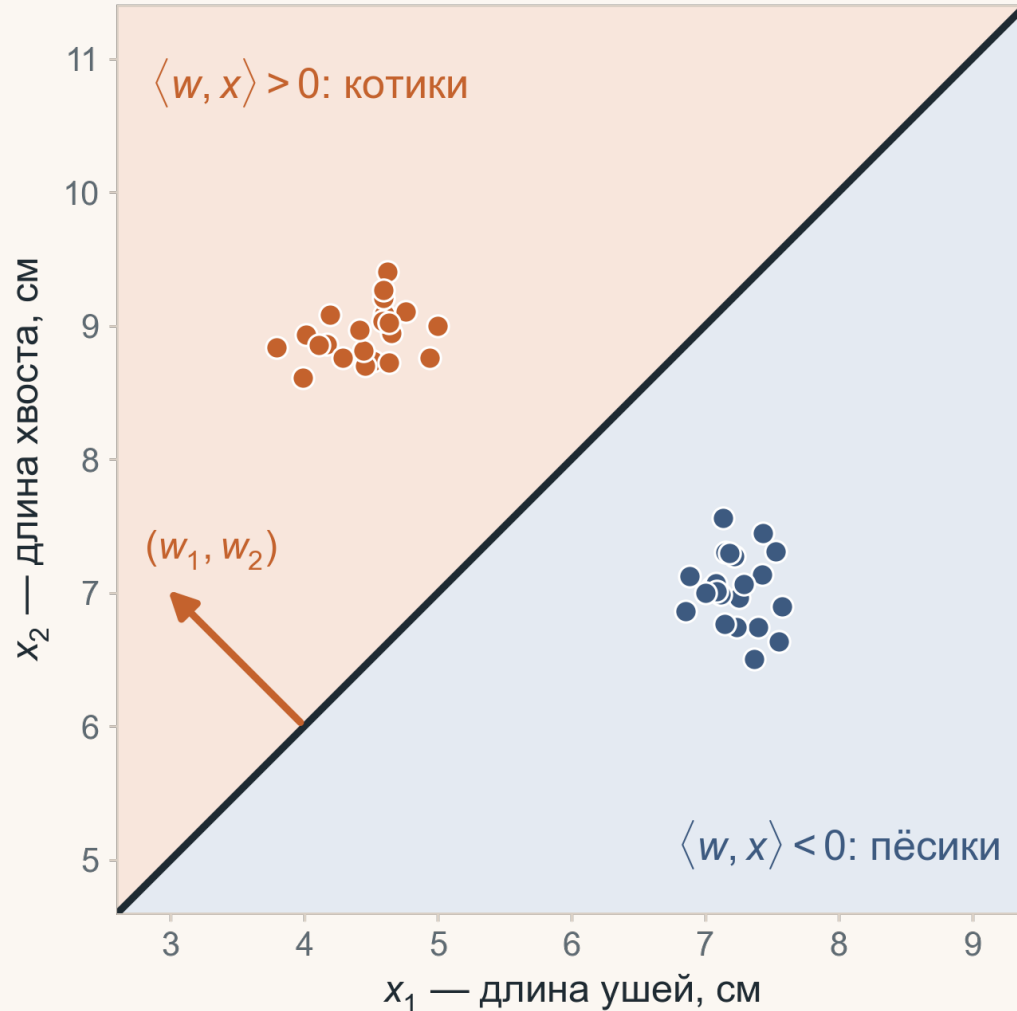
Котик (5, 9): $-2 + (-1) \cdot 5 + 1 \cdot 9 = +2$ — знак плюс.

Пёсик (7, 7): $-2 + (-1) \cdot 7 + 1 \cdot 7 = -2$ — знак минус.

Одна формула на все точки: умножили признаки на веса и сложили.

Весы здесь взяты для примера. Как подбирать их по данным — дальше в лекции.

По какую сторону от прямой



Знак $\langle w, x \rangle$ показывает, по какую сторону от прямой лежит точка:

- $\langle w, x \rangle > 0$ — с той стороны, куда смотрит вектор (w_1, w_2) : котики;
- $\langle w, x \rangle < 0$ — с противоположной стороны: пёсики;
- $\langle w, x \rangle = 0$ — на самой прямой.

Модуль показывает, на сколько точка далеко от прямой. Расстояние считается по школьной формуле:

$$\rho(x) = \frac{|\langle w, x \rangle|}{\sqrt{w_1^2 + w_2^2}}$$

Линейный классификатор

$$a(x) = \text{sign} \langle w, x \rangle$$

sign — знак числа: $+1$ для положительных чисел и -1 для отрицательных.

Метки классов: котик — $y = +1$, пёсик — $y = -1$. Точки ровно на прямой встречаются редко, им можно приписать любой класс.

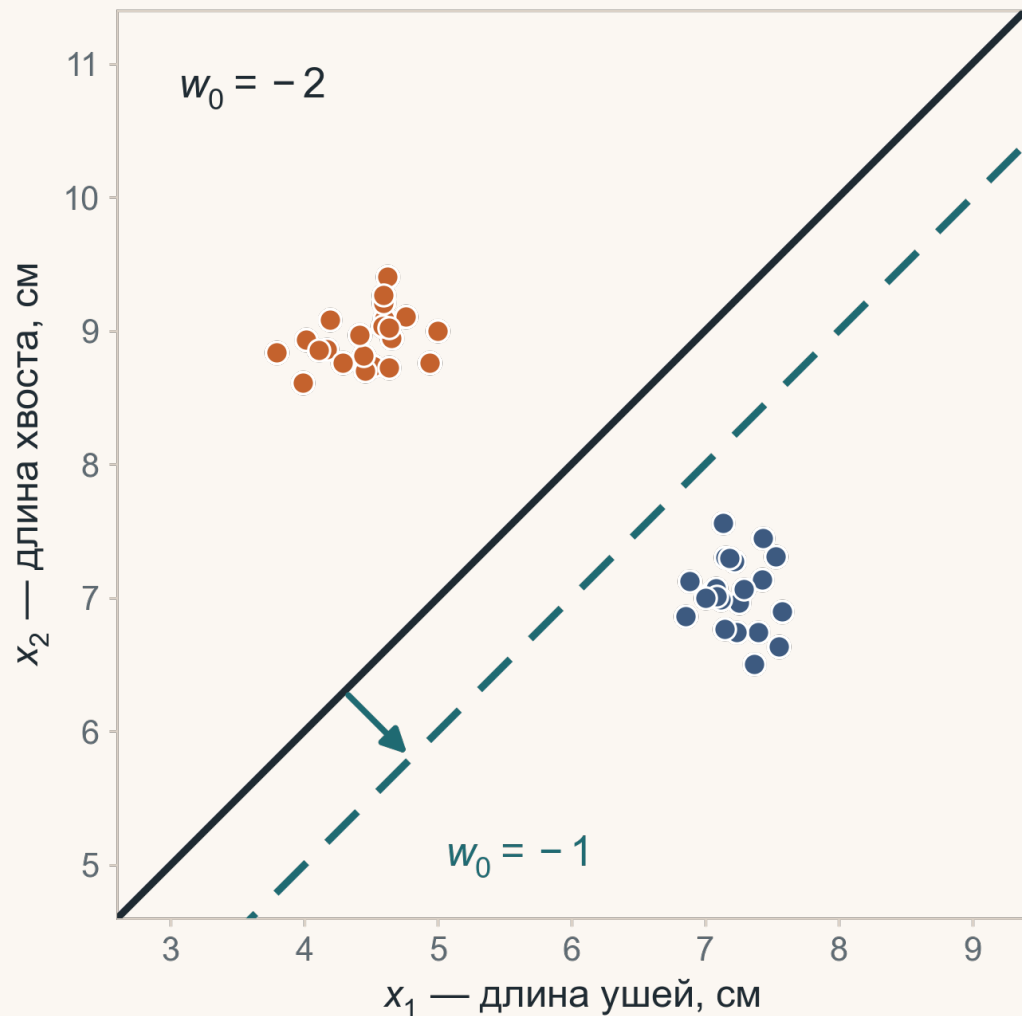
Признаков может быть сколько угодно:

$$\langle w, x \rangle = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Для трёх признаков граница — плоскость, для большего числа — гиперплоскость.

Обучить линейный классификатор — значит подобрать веса w .

Свободный член сдвигает прямую

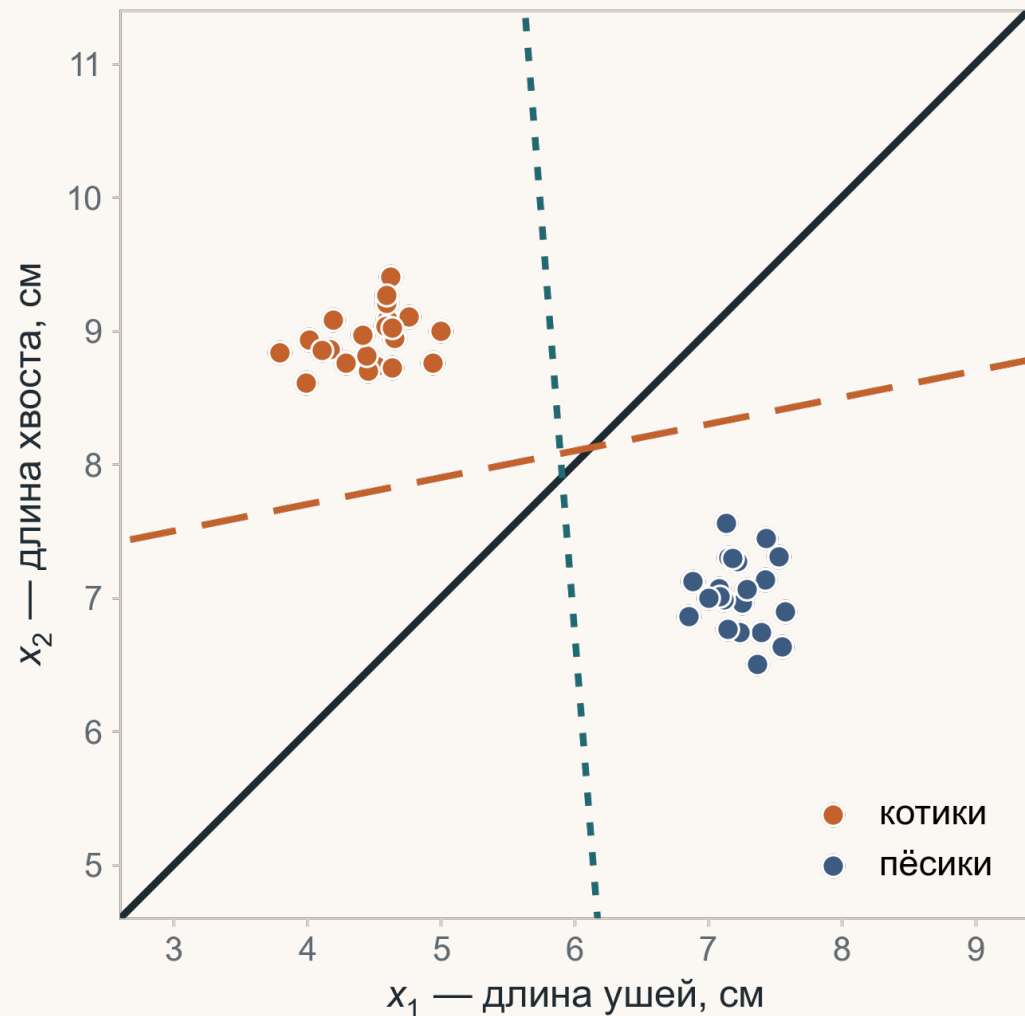


w_1 и w_2 задают наклон прямой, w_0 — её положение.

Увеличим w_0 с -2 до -1 : $\langle w, x \rangle$ вырастет на 1 у всех точек сразу, ведь $x_0 = 1$.

Прямая сдвинется параллельно себе — в сторону пёсиков, против вектора (w_1, w_2) . Область котиков станет шире.

Подходящих прямых много



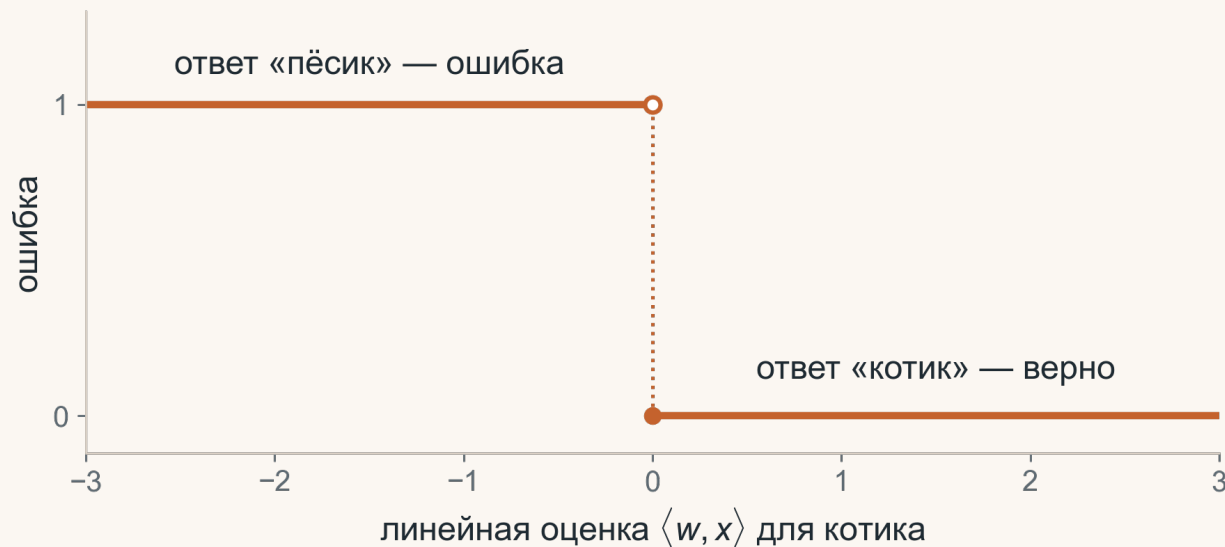
Все три прямые разделяют выборку без единой ошибки.

Какую выбрать? Нужна функция потерь, которую будем минимизировать, — как MSE в регрессии.

Самый очевидный вариант — доля ошибок. Но у неё есть серьёзная проблема.

Доля ошибок — ступенька

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} [a(x_i) \neq y_i] \rightarrow \min_w$$



[...] равно 1, если условие выполнено, и 0, если нет.

Чуть сдвинем веса — ответы почти не изменятся, и доля ошибок останется прежней. Потом она скачком изменится на $1/\ell$.

Градиент такой функции почти везде равен нулю — градиентный спуск не знает, куда шагать.

Выход: минимизировать гладкую замену доли ошибок. Первый такой способ — логистическая регрессия.

ПЕРВЫЙ СПОСОБ ВЫБРАТЬ ПРЯМУЮ

Логистическая регрессия

Модель выдаёт вероятность класса, а веса подбираются методом максимума правдоподобия.

Мало сказать «котик»

Знак $\langle w, x \rangle$ говорит только по какую сторону от прямой лежит точка. Но часто нужно знать, насколько модель уверена в ответе

Уверенный ответ

$$P(\text{котик}) = 0.98$$

Почти наверняка котик — фото можно сразу добавить в подборку котиков.

Сомнительный ответ

$$P(\text{котик}) = 0.55$$

Модель сомневается — такое фото лучше показать человеку для проверки.

Вероятности понадобятся и для обучения: из них построим функцию потерь.

Линейная оценка — ещё не вероятность

Линейная оценка $\langle w, x \rangle$

от $-\infty$ до $+\infty$

Любое число, например -7 , 0.3 или 4.2 .

Вероятность

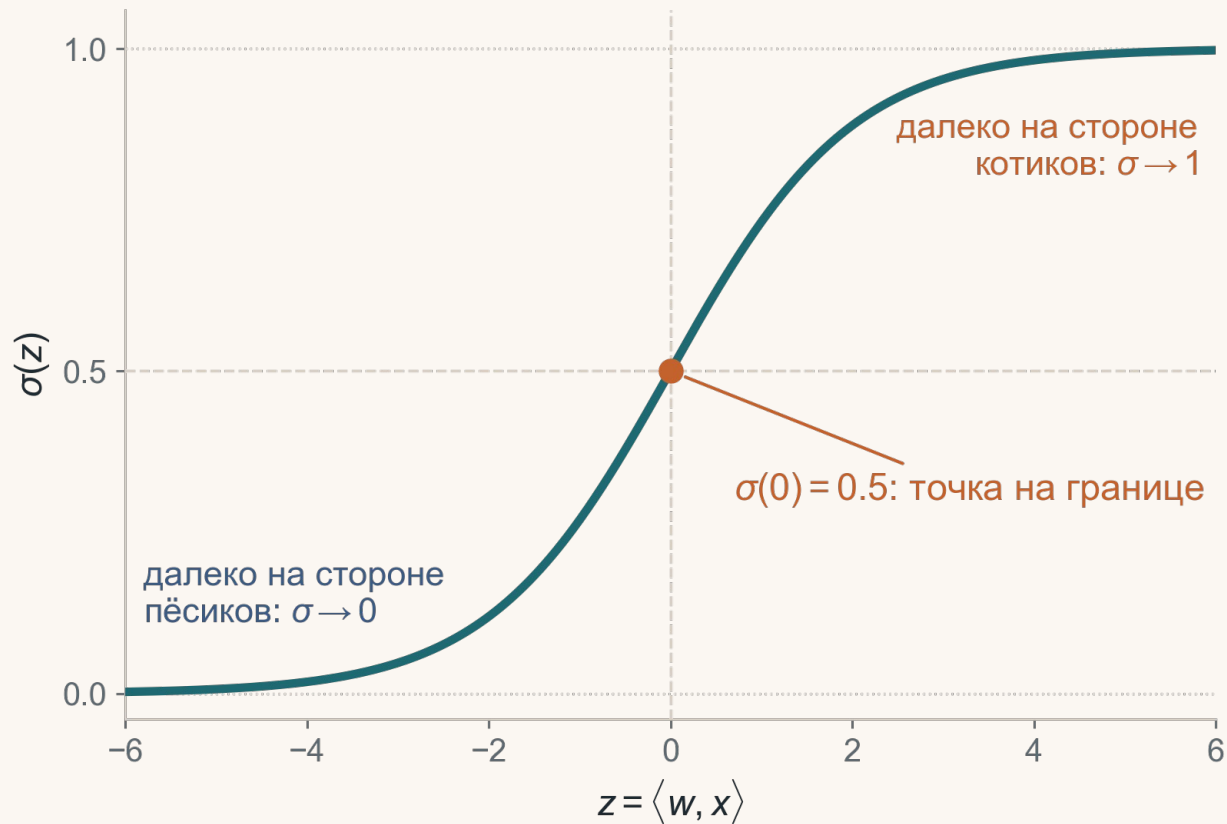
от 0 до 1

И ещё $P(\text{пёсик}) = 1 - P(\text{котик})$.

Нужна функция, которая переводит любое число в интервал от 0 до 1 и сохраняет порядок: чем больше оценка, тем выше вероятность котика.

Точке на прямой ($\langle w, x \rangle = 0$) естественно дать вероятность 0.5 — модель не знает, какой это класс.

Сигмоида



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(0) = 0.5$ — точка на прямой;
- чем дальше точка на стороне котиков, тем ближе σ к 1;
- чем дальше на стороне пёсиков, тем ближе к 0;
- функция гладкая и возрастающая.

Полезное свойство: $\sigma(-z) = 1 - \sigma(z)$.

Откуда берётся сигмоида

Шансы $p / (1 - p)$ — во сколько раз котик вероятнее пёсика: при $p = 0.8$ шансы равны 4 к 1. Шансы положительны, а их логарифм может быть любым числом — как и $\langle w, x \rangle$. Приравняем их:

$$\log \frac{p}{1-p} = \langle w, x \rangle$$

Выразим p :

$$\frac{p}{1-p} = e^{\langle w, x \rangle} \quad \Rightarrow \quad p = \frac{e^{\langle w, x \rangle}}{1 + e^{\langle w, x \rangle}} = \frac{1}{1 + e^{-\langle w, x \rangle}}$$

Получилась сигмоида: $p = \sigma(\langle w, x \rangle)$. Её не выбирали «на глаз»: она следует из предположения, что логарифм шансов линейно зависит от признаков.

Модель логистической регрессии

$$P(y = 1 \mid x) = \sigma(\langle w, x \rangle) = \frac{1}{1 + e^{-\langle w, x \rangle}}$$

- Метки классов теперь $y = 1$ (котик) и $y = 0$ (пёсик) — так удобнее записывать вероятности.
- Вероятность пёсика: $P(y = 0 \mid x) = 1 - \sigma(\langle w, x \rangle)$.
- Ответ «котик», если $P(y = 1 \mid x) > 0.5$, — это то же самое, что $\langle w, x \rangle > 0$. Граница остаётся прямой: логистическая регрессия — линейный классификатор.
- Функцию σ называют логистической, отсюда и название метода. Несмотря на слово «регрессия», это метод классификации.

Как подобрать веса: тот же рецепт ММП

На прошлой лекции MSE получилась из метода максимума правдоподобия (ММП).
Применим тот же рецепт к классификации.

1

Вероятность ответа

Модель даёт котика
вероятность $p_i = \sigma(\langle w, x_i \rangle)$, а
пёсику — $1 - p_i$.

2

Правдоподобие выборки

Объекты независимы,
поэтому вероятность всех
наблюдаемых ответов —
произведение.
Максимизируем его по w .

3

Минус логарифм

Произведение превращается
в сумму, максимум — в
минимум. Получаем функцию
потерь log-loss.

С какой вероятностью это котик: правдоподобие одного ответа

Модель «посмотрела» на данные и выдала $P = 0.92$.

Если действительно котик

0.92

Вероятность того, что мы видим, высокая — веса хорошие.

Если на самом деле пёсик

$1 - 0.92 = 0.08$

Модель считала правильный ответ маловероятным — веса плохие.

Хорошие веса дают высокую вероятность тем ответам, которые действительно есть в данных.

Функция потерь log-loss

Ищем веса w . Данные X и y (признаки x_i и ответы y_i) зафиксированы. Веса входят в формулы через $p_i = \sigma(\langle w, x_i \rangle)$ — вероятность котика для i -го объекта.

Правдоподобие выборки: для котика ($y_i = 1$) остаётся p_i , для пёсика ($y_i = 0$) — $1 - p_i$

$$P(y \mid X, w) = \prod_{i=1}^{\ell} p_i^{y_i} (1 - p_i)^{1 - y_i} \rightarrow \max_w$$

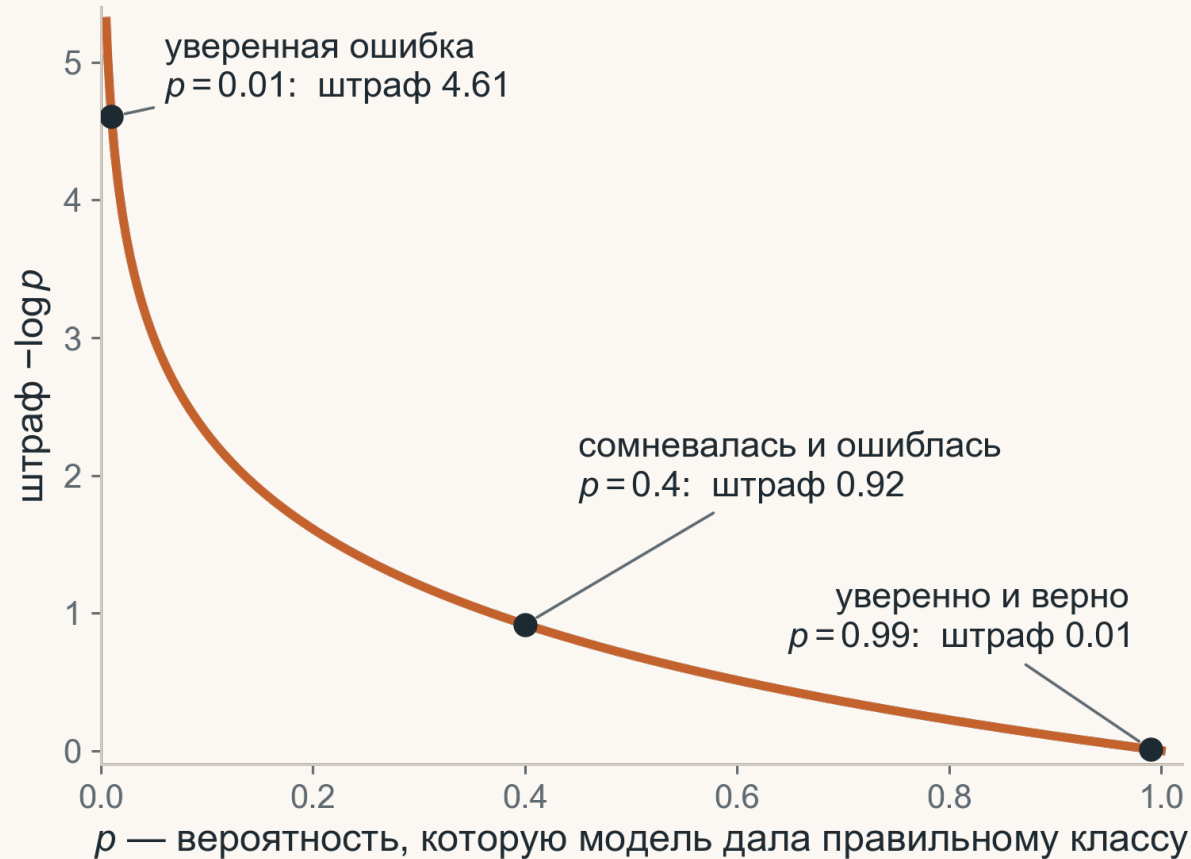
Берём минус логарифм: произведение становится суммой, а точка максимума — точкой минимума

$$Q(w) = - \sum_{i=1}^{\ell} (y_i \log p_i + (1 - y_i) \log(1 - p_i)) \rightarrow \min_w$$

- Меняем w — меняются все p_i , а с ними и $Q(w)$: это функция весов, а не данных.

$\log$ — натуральный логарифм. Другое название log-loss — бинарная кросс-энтропия.

Уверенная ошибка стоит дороже



Штраф за объект — $-\log p$, где p — вероятность, которую модель дала правильному классу: p_i у котика, $1 - p_i$ у пёсика.

- Сказала «котик» с вероятностью 0.99, а это пёсик: $p = 0.01$, штраф 4.6.
- Сказала «котик» с вероятностью 0.6, а это пёсик: $p = 0.4$, штраф 0.9.
- Сказала «котик» с вероятностью 0.99, и это котик: штраф 0.01.

Сильнее всего наказывается уверенность в неправильном ответе.

В отличие от доли ошибок, log-loss гладко зависит от весов — у него есть градиент.

Обучение: градиентный спуск

$$\nabla Q(w) = \sum_{i=1}^{\ell} (p_i - y_i) x_i \qquad w^{(k)} = w^{(k-1)} - \eta \nabla Q(w^{(k-1)})$$

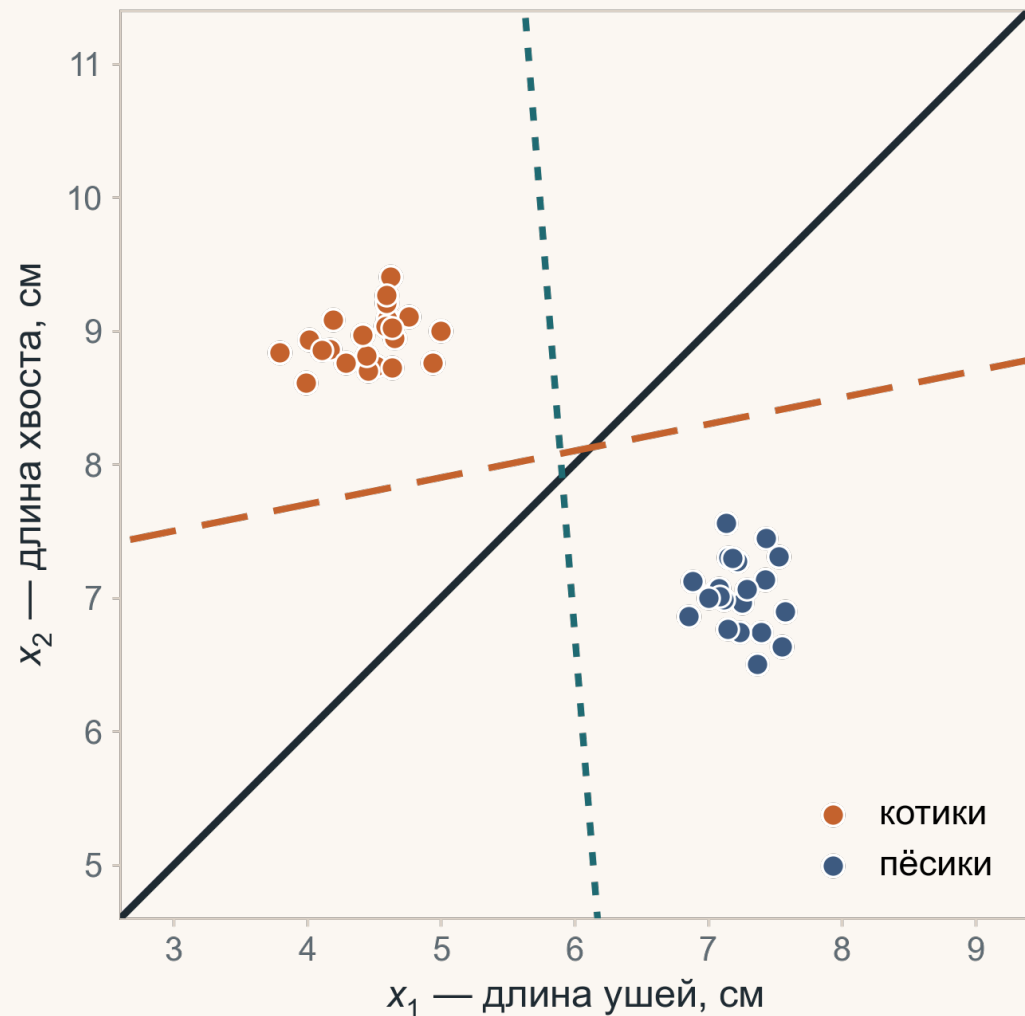
- Формулы для весов в явном виде, как у МНК, здесь нет — минимум ищем градиентным спуском (или SGD и mini-batch с прошлой лекции).
- $p_i = \sigma(\langle w, x_i \rangle)$ считаем при текущих весах, $p_i - y_i$ — ошибка модели в вероятности. Котик ($y_i = 1$) с $p_i = 0.2$ даёт -0.8 : шаг увеличит $\langle w, x_i \rangle$, и вероятность котика вырастет.
- η — длина шага (learning rate), x_i — вектор признаков с $x_0 = 1$.
- Log-loss — выпуклая и гладкая функция весов: как и у MSE, «ложных» локальных минимумов у неё нет.

ВТОРОЙ СПОСОБ ВЫБРАТЬ ПРЯМУЮ

Метод опорных векторов (SVM)

Среди прямых, разделяющих классы, выбираем ту, что проходит как можно дальше от точек обоих классов.

Какая прямая надёжнее?

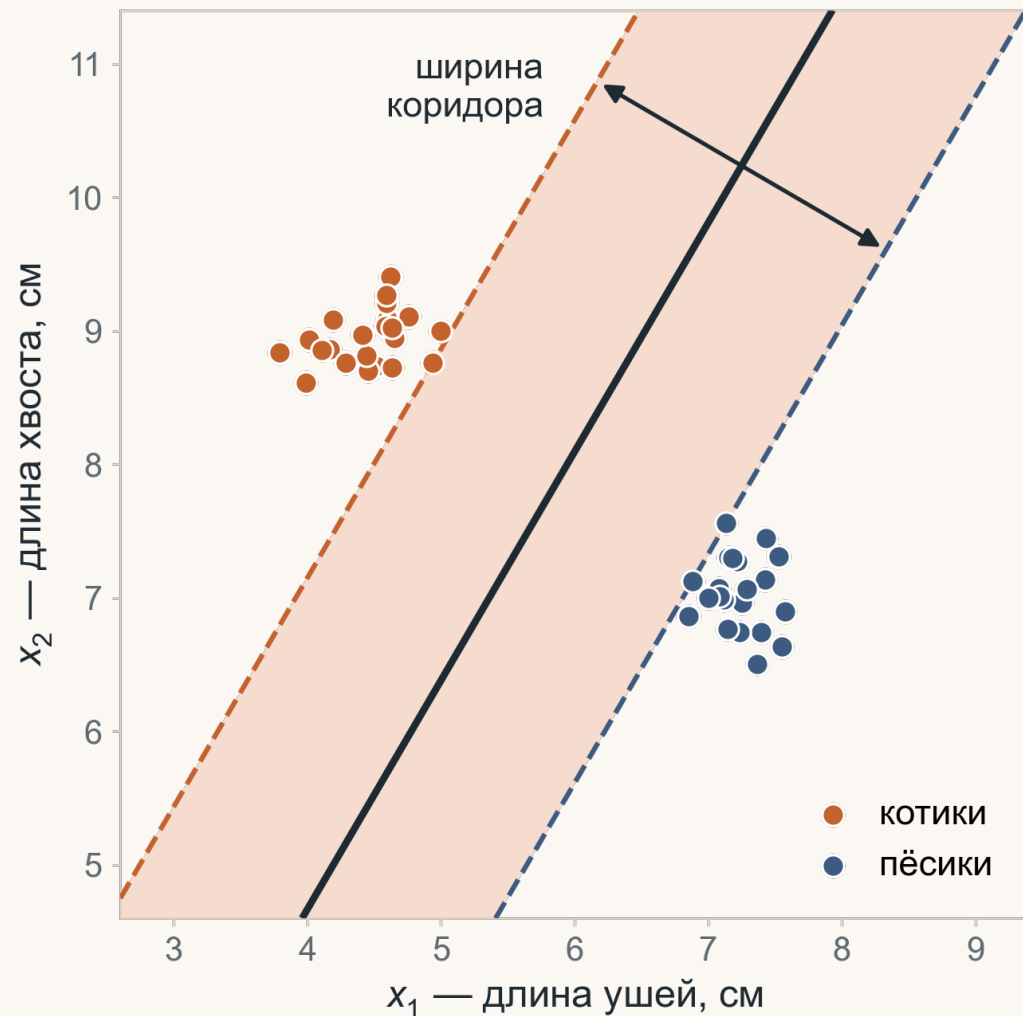


Все три прямые разделяют выборку без ошибок — по доле ошибок они равны.

Но прямая, проходящая вплотную к точкам, легко ошибётся на новом животном, которое чуть отличается от уже виденных.

Надёжнее прямая с наибольшим запасом до ближайших точек обоих классов.

Самый широкий коридор



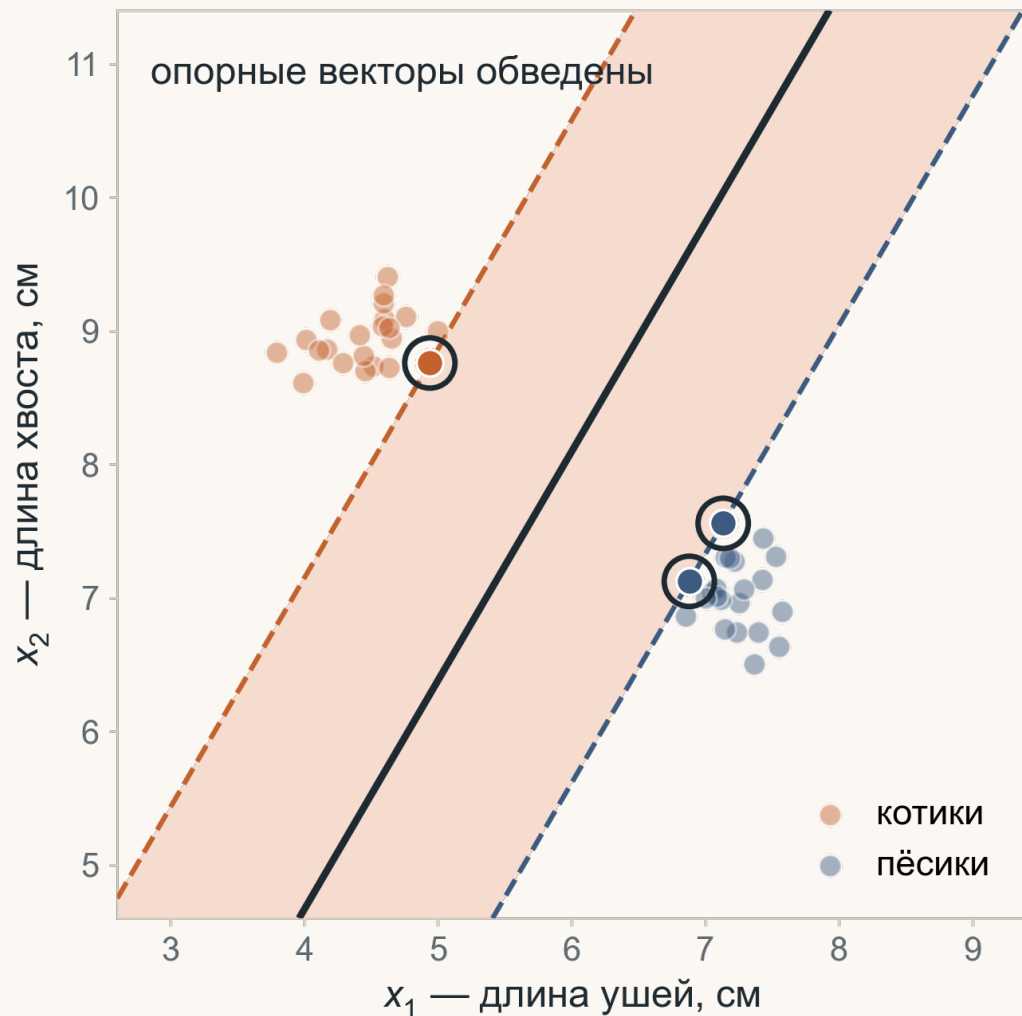
Коридор — полоса вокруг прямой, внутри которой нет точек выборки.

SVM выбирает прямую с самым широким коридором.

Прямая проходит ровно посередине: ближайший котик и ближайший пёсик находятся от неё на одинаковом расстоянии.

Широкий коридор — запас прочности: новой точке нужно сильно отклониться, чтобы попасть на чужую сторону.

Опорные векторы



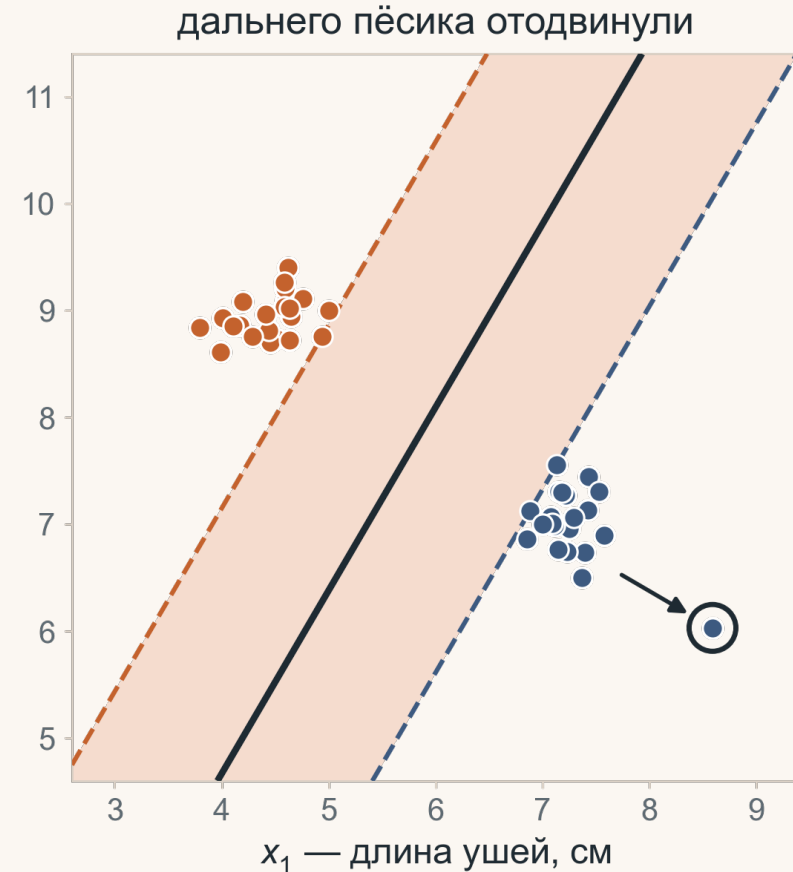
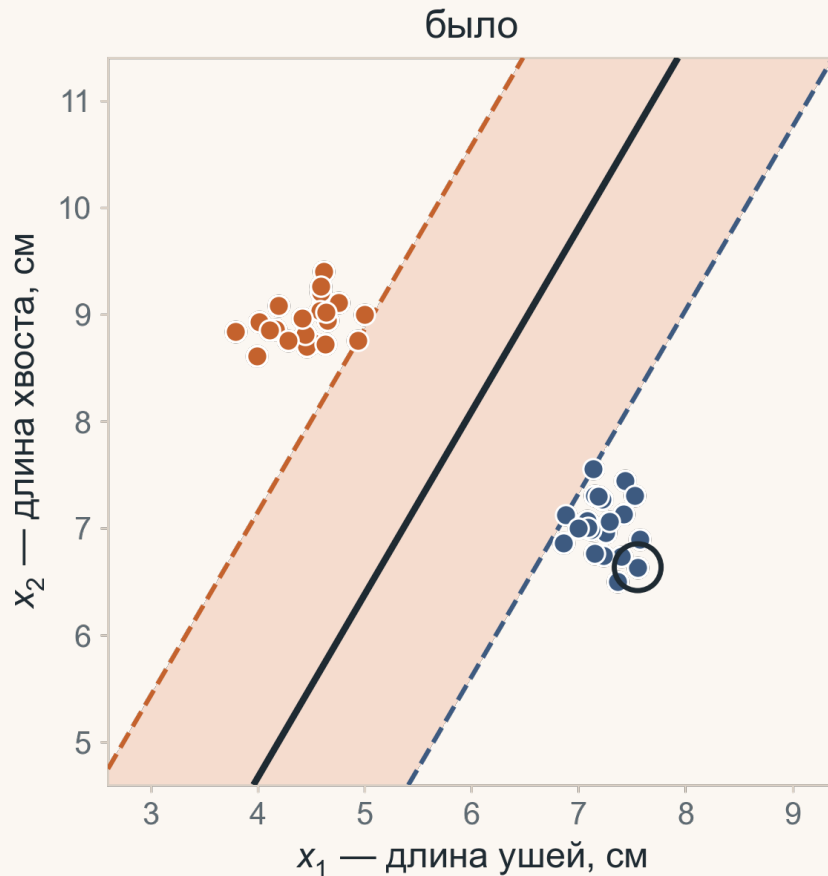
Края коридора упираются в ближайшие точки выборки. Эти точки называют **опорными векторами** (support vectors).

Решение SVM зависит только от них: остальные точки можно убрать — прямая не изменится.

Отсюда и название метода. В sklearn опорные векторы лежат в атрибуте `support_vectors_`.

Дальние точки не влияют

Отодвинем пёсика, который не был опорным, ещё дальше — коридор и прямая не изменятся. А вот сдвиг опорной точки изменил бы решение.



Ширина коридора

Умножим все веса на одно положительное число — прямая не изменится. Выберем масштаб так, чтобы на краях коридора было $\langle w, x \rangle = \pm 1$. По формуле расстояния край отстоит от прямой на $1 / \|w\|$, поэтому ширина коридора равна

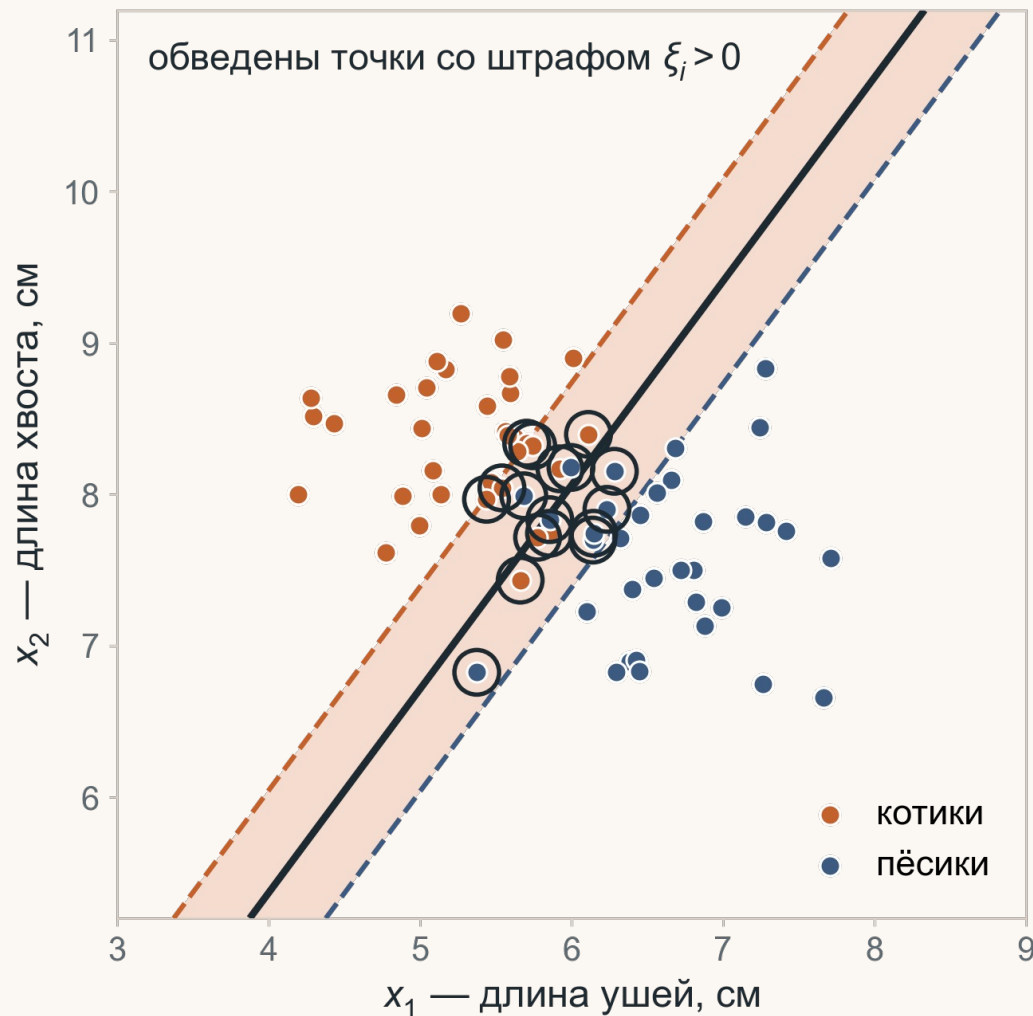
$$\frac{2}{\|w\|}, \quad \|w\| = \sqrt{w_1^2 + w_2^2}$$

w_0 в длину вектора не входит: он сдвигает прямую, но не меняет ширину. Чем меньше $\|w\|$, тем шире коридор. Задача SVM:

$$\frac{1}{2} \|w\|^2 \rightarrow \min_w, \quad y_i \langle w, x_i \rangle \geq 1, \quad i = 1, \dots, \ell$$

Метки снова $y_i = +1$ (котик) и $y_i = -1$ (пёсик). Условие $y_i \langle w, x_i \rangle \geq 1$ значит: точка на своей стороне и не внутри коридора.

Классы перемешаны



В реальных данных классы пересекаются: коридора без точек внутри не существует.

Разрешим точкам заходить в коридор и даже на чужую сторону, но за каждое нарушение назначим штраф $\xi_i \geq 0$.

Это SVM с **мягким зазором** (soft margin).
Вариант без нарушений называют жёстким зазором (hard margin).

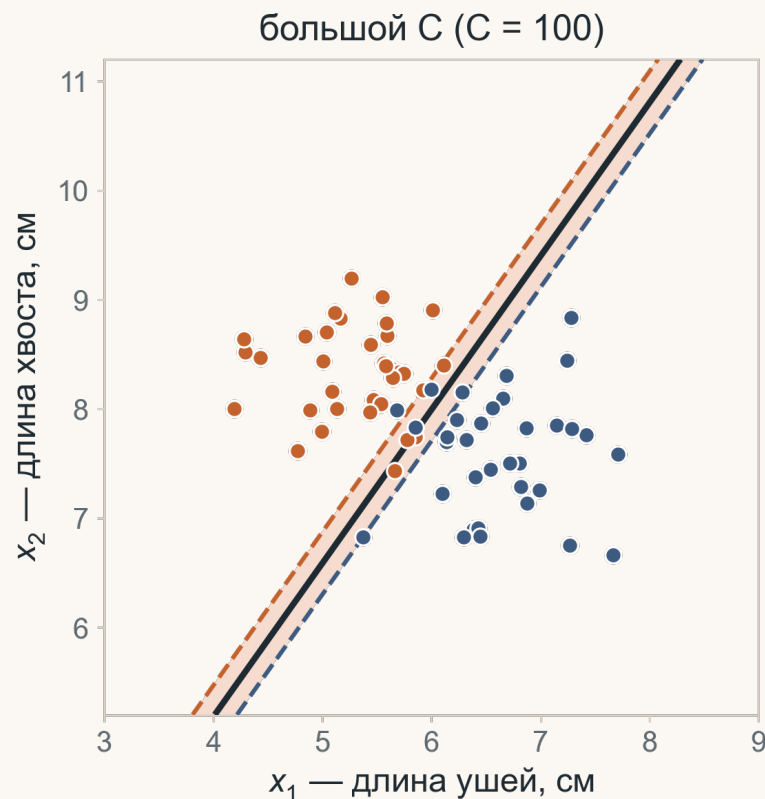
Что минимизирует SVM

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^{\ell} \xi_i \rightarrow \min_w \quad \xi_i = \max(0, 1 - y_i \langle w, x_i \rangle)$$

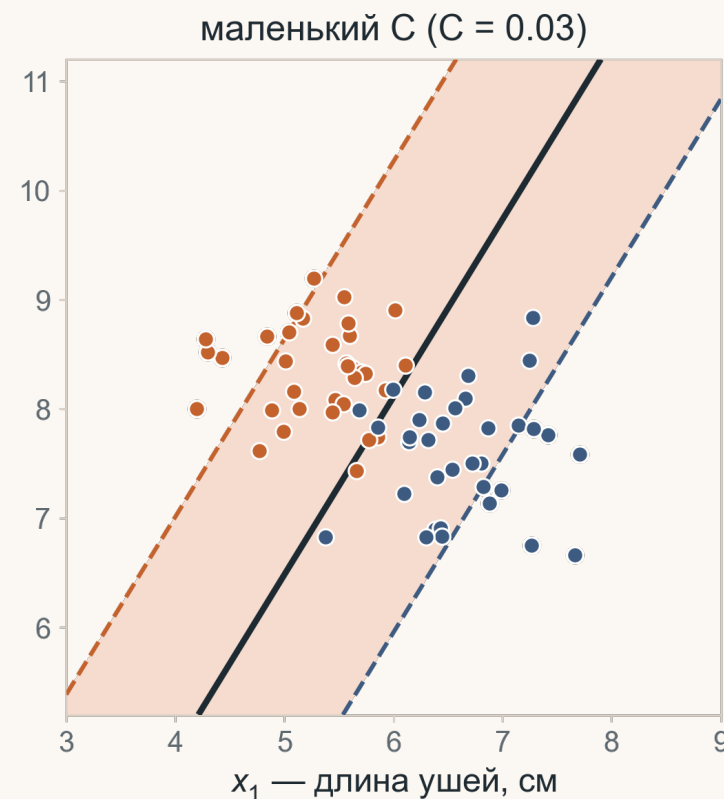
- $\frac{1}{2}\|w\|^2$ — чем меньше, тем шире коридор;
- ξ_i — штраф i -го объекта: 0 — объект на своей стороне и не заходит в коридор; от 0 до 1 — внутри коридора, но на своей стороне; больше 1 — на чужой стороне (ошибка);
- $C > 0$ — гиперпараметр: насколько штрафы важнее ширины коридора.

Функцию $\max(0, 1 - y_i \langle w, x_i \rangle)$ называют hinge loss. Как и раньше, $\|w\|^2 = w_1^2 + w_2^2$, без w_0 .

Параметр C



Большой C — нарушения дорогие. Коридор узкий, граница подстраивается под отдельные точки и может переобучиться.



Маленький C — важнее ширина. Модель терпит точки в коридоре и даже ошибки, зато граница устойчивее к выбросам.

Логистическая регрессия и SVM

Логистическая регрессия

SVM

Что выдаёт

вероятность $P(y = 1 \mid x)$

класс и оценку $\langle w, x \rangle$ — это не вероятность

Как выбирает прямую

максимум правдоподобия, то есть минимум log-loss

самый широкий коридор с учётом штрафов

Какие объекты влияют

все объекты выборки

только опорные векторы

Когда удобен

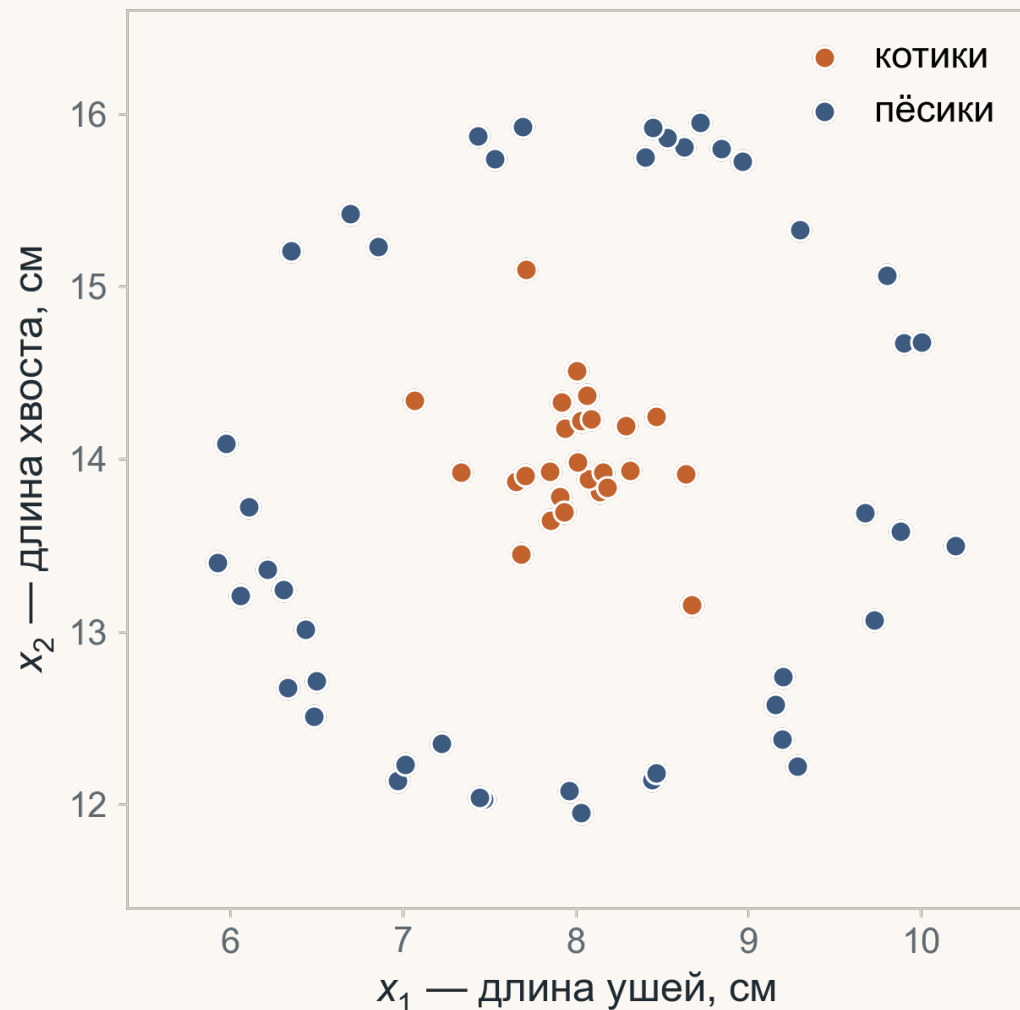
нужны вероятности и понятные веса

нужна граница с запасом; можно применить ядра

КОГДА ДАННЫЕ ЛИНЕЙНО НЕ РАЗДЕЛИМЫ

Ядра: как согнуть границу

Когда прямой мало



Котики в центре, пёсики вокруг — кольцом.

Ни одна прямая не отделит центр от кольца: ни логистическая регрессия, ни линейный SVM здесь не помогут.

Среди класса линейных моделей подходящей границы просто нет.

Идея: менять не модель, а признаки.

Добавим новый признак

$$r^2 = (x_1 - c_1)^2 + (x_2 - c_2)^2$$

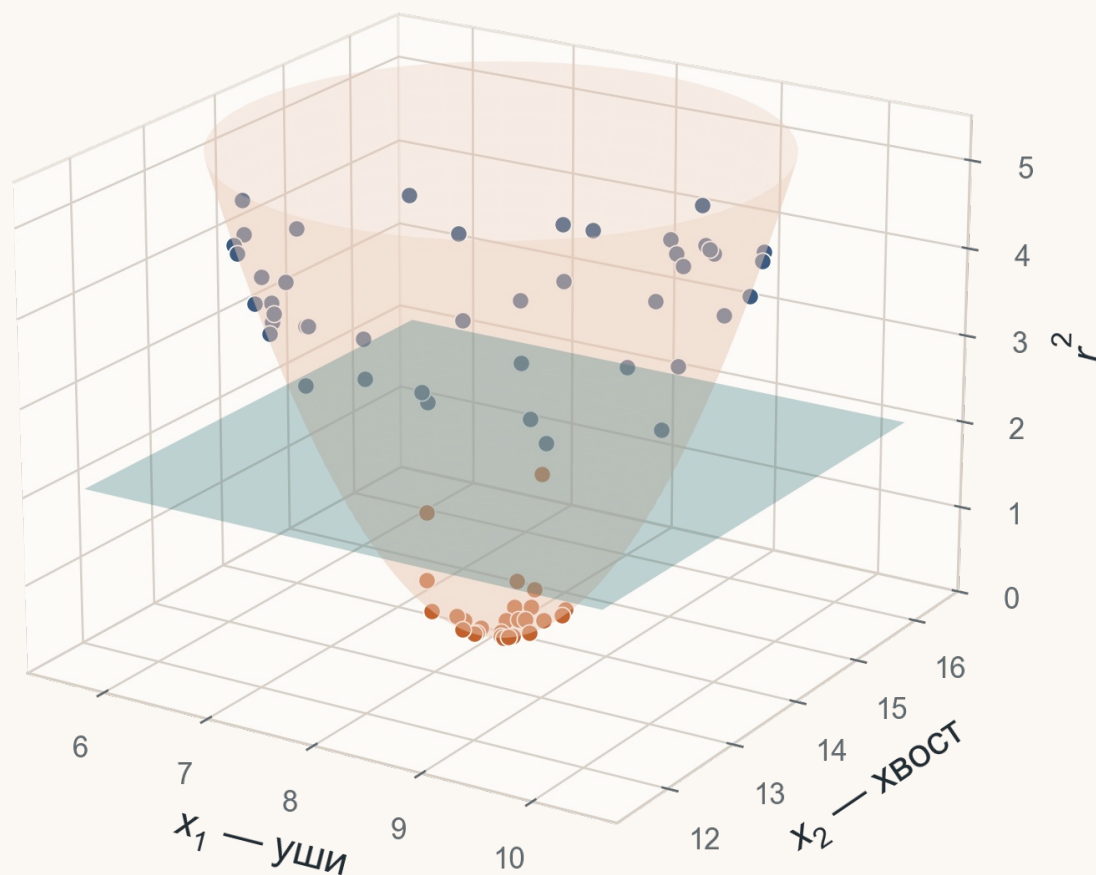
$c = (8, 14)$ — центр облака котиков, r^2 — квадрат расстояния от животного до центра. У котиков r^2 маленький, у пёсиков на кольце — большой.

Модель остаётся линейной по весам:

$$a(x) = \text{sign}(w_0 + w_1x_1 + w_2x_2 + w_3r^2)$$

Центр здесь выбран «на глаз», для наглядности. Можно вместо r^2 добавить признак $x_1^2 + x_2^2$ — тогда центр окружности подберут веса при x_1 и x_2 .

Поднимаем точки в 3D

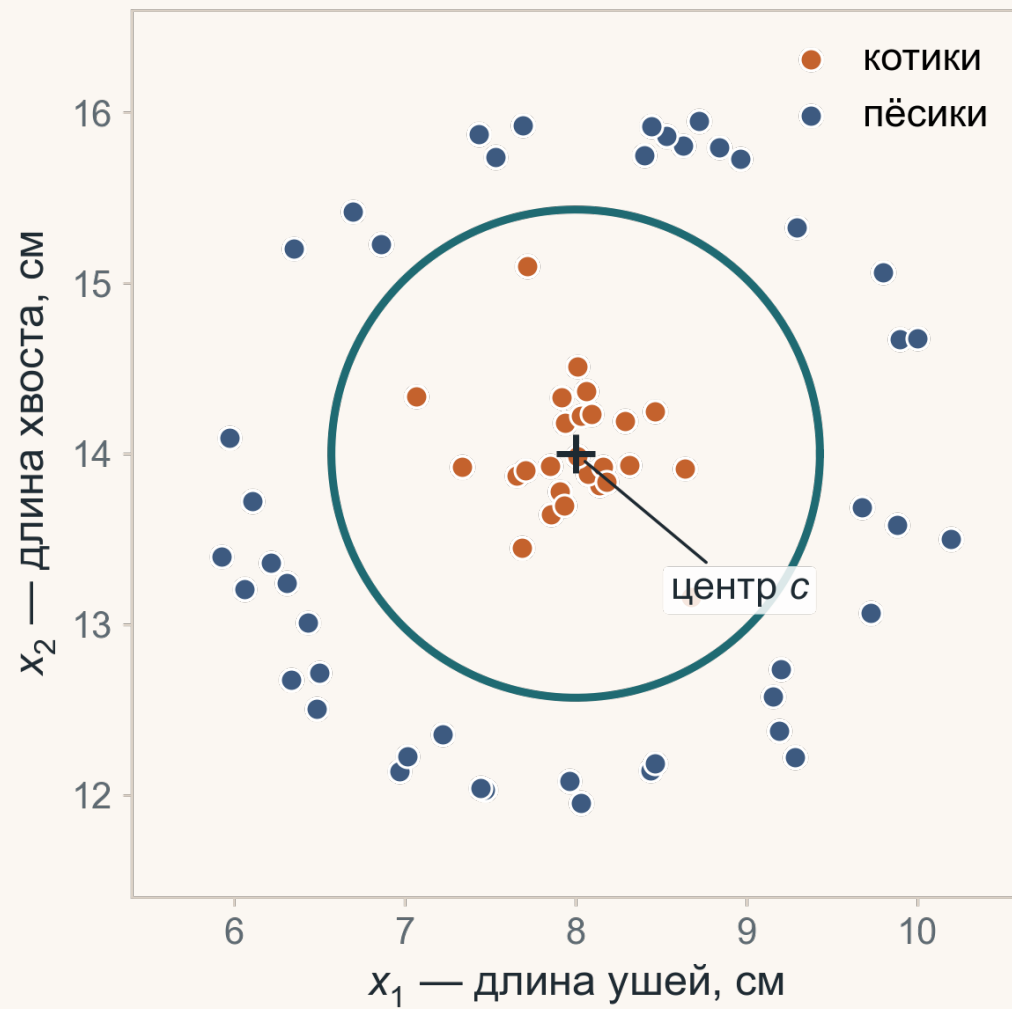


Отложим новый признак r^2 по третьей оси.

Котики оказываются внизу, пёсики — высоко.

Теперь классы разделяет плоскость — обычная линейная модель, только признаков три.

Плоскость в 3D — окружность на плоскости



Пусть разделяющая плоскость горизонтальна:
 $w_1 = w_2 = 0$. Тогда граница:

$$w_0 + w_3 r^2 = 0 \quad \Rightarrow \quad r^2 = -\frac{w_0}{w_3}$$

Все точки границы находятся на одном расстоянии от центра c — это окружность.

**Линейная граница в новых признаках —
нелинейная граница в исходных.**

Новые признаки можно не вычислять

Хорошие признаки $\varphi(x)$ бывают очень длинными: все попарные произведения, степени признаков... — тысячи координат, а иногда и бесконечно много.

У SVM есть полезное свойство: при обучении и предсказании объекты участвуют только через скалярные произведения пар объектов $\langle x_i, x_j \rangle$. Это следует из так называемой двойственной задачи — выводить её не будем.

Значит, достаточно уметь быстро считать $\langle \varphi(x), \varphi(z) \rangle$ — сами новые признаки строить не нужно. Это и есть **kernel trick**.

Kernel trick

Ядро — функция от пары объектов

$$K(x, z) = \langle \varphi(x), \varphi(z) \rangle$$

$\varphi(x)$ — новые признаки объекта, $K(x, z)$ — их скалярное произведение, вычисленное прямо по исходным признакам.

Пример: квадратичное ядро

$$\langle x, z \rangle^2 = (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = \langle \varphi(x), \varphi(z) \rangle$$

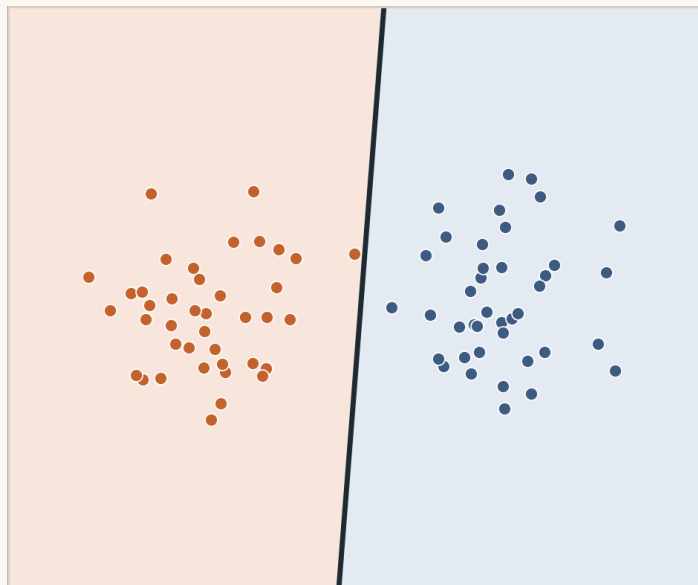
$$\varphi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$$

Слева — одно скалярное произведение в исходных признаках и возведение в квадрат. Справа — то же число через три новых признака. Для $\langle x, z \rangle^d$ с большим d и многих признаков экономия огромная.

Популярные ядра

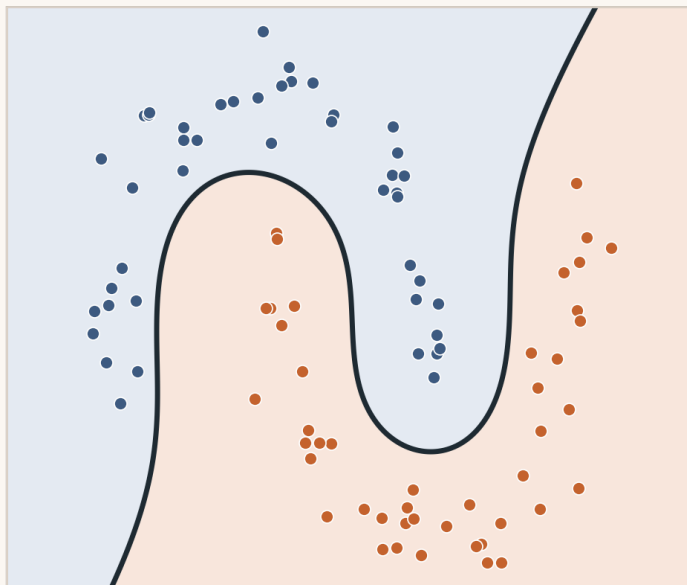
линейное

$$K(x, z) = \langle x, z \rangle$$



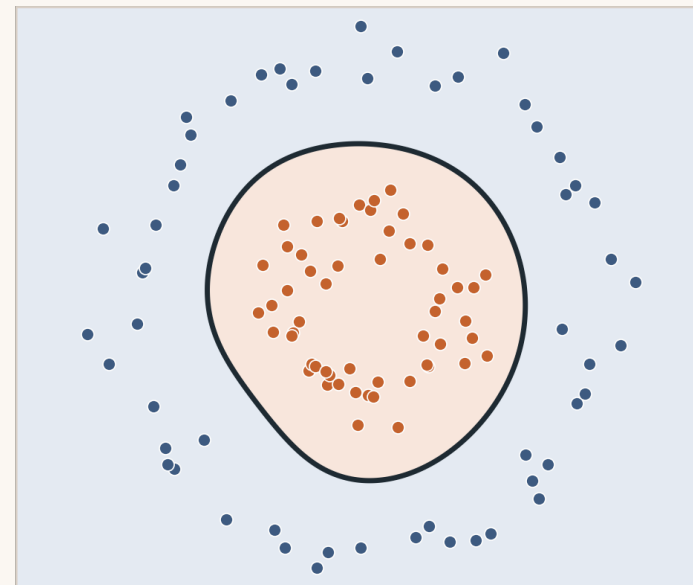
полиномиальное, $d = 3$

$$K(x, z) = (\langle x, z \rangle + 1)^d$$



RBF (гауссово), $\gamma = 1$

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$



- Полиномиальное ядро даёт границы-многочлены степени d .
- RBF измеряет похожесть: у близких точек $K \approx 1$, у далёких $K \approx 0$. Чем больше γ , тем извилистее граница и тем легче переобучиться.

d и γ — гиперпараметры, как и C : их подбирают на валидации.

Чем платим за ядра

С ядрами

- + гибкая нелинейная граница
- по весам уже не прочесть, какой исходный признак за что отвечает
- нужны значения ядра для всех пар объектов: на сотнях тысяч объектов обучение идёт относительно долго
- больше гиперпараметров: C , γ , d

Линейная модель

- + вес каждого признака можно объяснить
- + быстро обучается даже на миллионах объектов
- граница — только прямая (гиперплоскость)

В кредитном скоринге и медицине часто важнее объяснимость — там выбирают линейные модели.

Что вызвать в sklearn

Логистическая регрессия

`predict` — класс, `predict_proba` —
вероятность

```
LogisticRegression()
```

Линейный SVM

`predict` — класс, `decision_function` —
оценка $\langle w, x \rangle$

```
SVC(kernel="linear", C=1.0)
```

SVM с ядром

то же, но граница нелинейная

```
SVC(kernel="rbf", C=1.0, gamma="scale")
```

Масштаб признаков

стандартизация внутри пайплайна

```
make_pipeline(StandardScaler(), SVC())
```

КАК ИЗМЕРИТЬ КАЧЕСТВО

Метрики качества

Accuracy, матрица ошибок, precision и recall, порог и ROC-AUC.

Ассурасу 95% — и ни одного котика

95%

ассурасу — доля верных
ответов

В выборке 950 пёсиков и 50 котиков.

Модель на все фото отвечает «пёсик».

Верных ответов 950 из 1000 — ассурасу 95%. Но ни одного котика модель не нашла.

Когда один класс сильно преобладает (дисбаланс классов), ассурасу почти ничего не говорит о качестве.

Матрица ошибок

	на самом деле котик	на самом деле пёсик
модель: «КОТИК»	TP = 30 верно назвали котиком	FP = 10 зря назвали котиком
модель: «пёсик»	FN = 20 пропустили котика	TN = 940 верно назвали пёсиком
$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$		

Положительный класс — котик: его мы ищем.

Всего 1000 животных: 50 котиков и 950 пёсиков.

Ассурасу нашей модели:
 $(30 + 940) / 1000 = 97\%$.

Модель «всегда пёсик»: TP = 0, FP = 0, FN = 50, TN = 950 — ассурасу 95%.

Precision — точность

$$\text{precision} = \frac{TP}{TP + FP}$$

Какая доля тех, кого модель назвала котиками, — действительно котики.

Модель назвала котиками 40 животных, из них 30 — котики.

Низкая precision — в подборке котиков много пёсиков (ложных срабатываний).

Наша модель

TP = 30

FP = 10

$$30 / 40 = 0.75$$

Recall — полнота

$$\text{recall} = \frac{TP}{TP + FN}$$

Какую долю всех котиков модель нашла.

Котиков 50, модель нашла 30.

Низкий recall — многих котиков модель пропустила.

У модели «всегда пёсик» recall = 0, хотя accuracy у неё 95%.

Наша модель

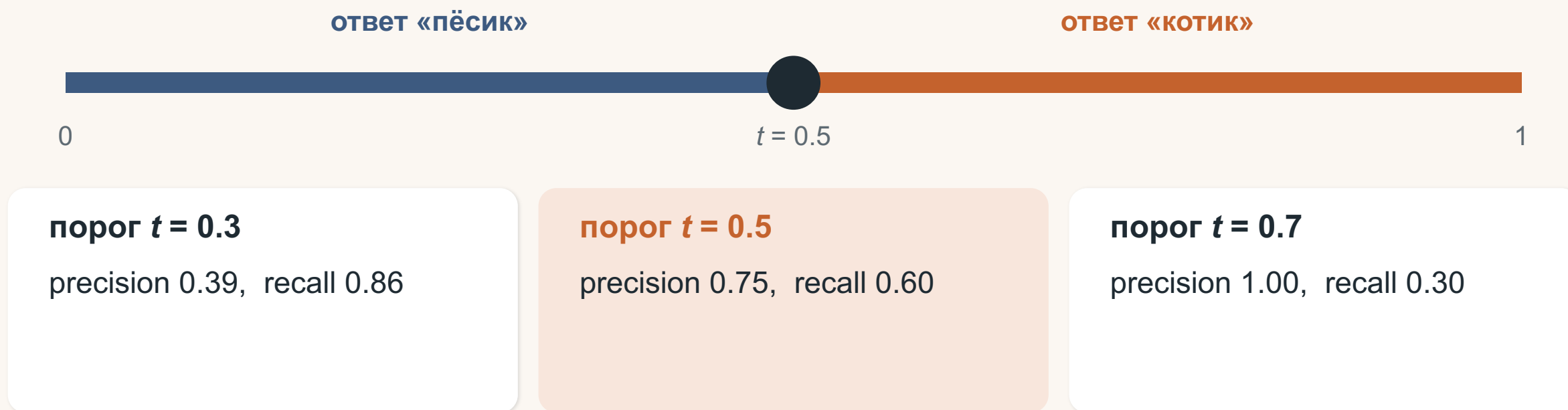
TP = 30

FN = 20

$$30 / 50 = 0.6$$

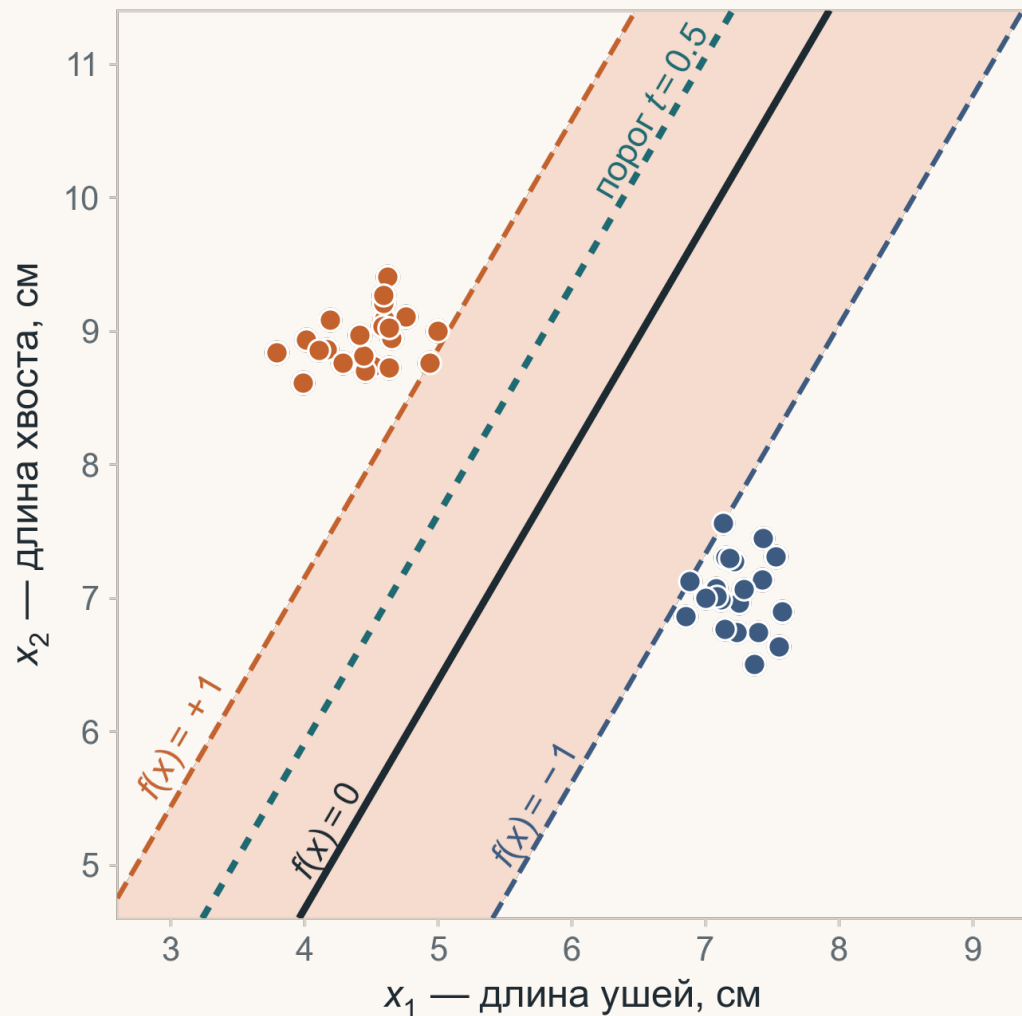
Порог у логистической регрессии

Логистическая регрессия выдаёт вероятность, а класс получаем сравнением с порогом t : «котик», если $P(y = 1 \mid x) > t$. По умолчанию $t = 0.5$.



Выше порог — «котик» говорим реже и осторожнее: precision обычно растёт, recall падает. Порог выбирают после обучения, исходя из задачи: что дороже — ложная тревога или пропущенный котик?

Порог у SVM



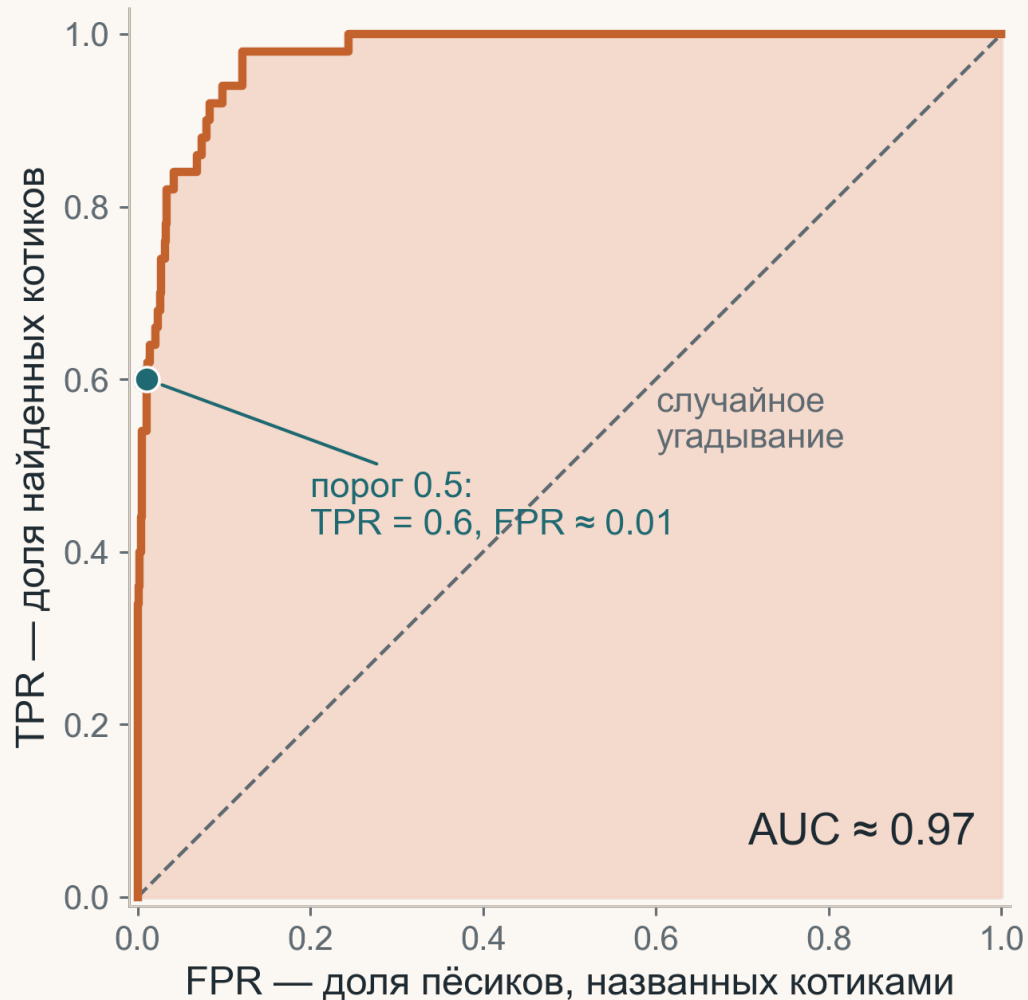
SVM выдаёт не вероятность, а оценку $f(x) = \langle w, x \rangle$ (в sklearn — `decision_function`).

«Котик», если $f(x) > t$. По умолчанию $t = 0$ — это сама разделяющая прямая, а края коридора — уровни $f(x) = \pm 1$.

Меняя t , сдвигаем границу параллельно. Больше t — «котик» говорим реже.

Не путайте порог и C : C задают до обучения, порог выбирают после.

ROC-кривая и AUC



Перебираем все пороги t и для каждого считаем две доли:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

TPR — это recall. FPR — доля пёсиков, которых модель ошибочно назвала котиками.

Каждый порог даёт точку (FPR, TPR), все вместе — ROC-кривую.

AUC — площадь под ней: 1 — идеальное упорядочивание, 0.5 — случайное угадывание.

AUC — вероятность того, что случайный котик получит оценку выше случайного пёсика. От порога она не зависит.

Что мы обсудили

- 1 **Классификация** — ответ из конечного набора классов; решающая граница делит пространство признаков на области классов.
- 2 **Линейный классификатор** — $a(x) = \text{sign}\langle w, x \rangle$: граница — прямая (гиперплоскость), знак показывает сторону.
- 3 **Логистическая регрессия** — $P(y = 1 \mid x) = \sigma(\langle w, x \rangle)$; веса находим методом максимума правдоподобия, то есть минимизируем log-loss.
- 4 **SVM** — самый широкий коридор; C балансирует ширину и штрафы, решение задают опорные векторы.
- 5 **Ядра** — новые признаки делают границу нелинейной; kernel trick считает $\langle \phi(x), \phi(z) \rangle$, не вычисляя ϕ .
- 6 **Метрики** — accuracy обманывает при дисбалансе классов; precision и recall зависят от порога, ROC-AUC — нет.